Actividad 6 Entrega final del proyecto.

Yolanda Gonzalez Herrera
(100129489)
Santiago León Duran
(100151578)
Jhon Sebastian Rodriguez Contreras
(100159126)
Jeison Orlando baquero Gutierrez
(100130741)

Facultad de Ingeniería

Corporación Universitaria Iberoamericana

Proyecto de Software

Tatiana Cabrera

2025

Tabla de contenido

Fase de Planificación. 4

1. Contextualización de la necesidad. 4
2. Planteamiento del problema. 4
3. Alcance del proyecto. (restricciones, criterios de aceptación del proyecto). 5
4. Estructura del Desglose. (EDT) 5
5. Objetivos 6
6. Metodología ágil seleccionada (https://n9.cl/7z1xoa) 7
7. Justificación alcance corto, medio, largo), descripciones soluciones al problema, respuesta a los stakeholders. 7
8. Mapa de Stakeholders y clasificación de los mismos 8
9. Matriz de riesgos. 9
10. Presupuesto. 10
11. Levantamiento de información (herramientas y conclusiones) 10
12. Diagrama de flujo de solución selecciona. 12
13. Definición de RQF-RQNF manejar un formato para la documentación. 15

ACTIVIDAD 4 –DISEÑO DE PROTOTIPO 16

1. Prototipo funcional 16

Link 16

2. Casos de uso documentados 17
3. Diagrama de secuencia 31
4. Diagrama de clases 33
5. Diagrama de componentes 34
6. Diagrama de despliegue 34
7. Modelo de base de datos 35
8. Pruebas de Usabilidad: 40

Documentación de la Arquitectura de Software 46

1. Justificación de la Arquitectura de Software 46
2. Implementación de la Arquitectura por Capas 47
3. Flujo de Desarrollo e Integración Continua (CI/CD) 47
4. Elementos Documentados en el Repositorio del Grupo 48
5. Documentación Complementaria en el Documento del Proyecto 49
6. Conclusión 49

Información general 50

Pruebas unitarias (backend) 50

Módulo de Autenticación 51

Módulo de Tareas 52

Pruebas de integración 55
Pruebas de sistema 57
Pruebas funcionales 59
Pruebas no funcionales 61
Pruebas de Rendimiento 61
Proceso de despliegue detallado 62
..... 62
FASE 1: Preparación local. 62
FASE 2: Configuración InfinityFree. 62
FASE 3: Base de datos. 62
FASE 4: Subida de archivos. 63

Fase de Planificación.

Link GitHub (https://github.com/jrodr337ibero/PROYECTO_SOFTWARE.git)

1. **Contextualización de la necesidad.**
   En entornos laborales y académicos, la gestión eficiente de tareas en equipo es fundamental para alcanzar objetivos comunes. Las herramientas tradicionales de listas de tareas no siempre permiten una colaboración fluida ni una visualización clara del progreso. Por ello, se requiere una aplicación que facilite la asignación de tareas, seguimiento en tiempo real y comunicación integrada para mejorar la productividad y coordinación del equipo.

2. **Planteamiento del problema.**
   En la actualidad, tanto a nivel personal como en entornos laborales, la gestión eficiente del tiempo y de las actividades se ha convertido en un desafío constante. Muchas personas y equipos enfrentan dificultades para organizar sus tareas, lo que genera problemas como:

   - Pérdida de información al anotar tareas en diferentes medios (papel, notas en el celular, correos electrónicos).
   - Falta de seguimiento en las actividades, ocasionando retrasos en la entrega de proyectos.
   - Dificultad para priorizar tareas urgentes frente a tareas secundarias.
   - Escasa colaboración entre miembros de un equipo, ya que no existen herramientas centralizadas para asignar, delegar o monitorear responsabilidades.
   - Baja productividad, debido a la desorganización y ausencia de recordatorios efectivos.

   Estas limitaciones afectan tanto a estudiantes, trabajadores independientes como a equipos de empresas, generando retrasos, duplicidad de esfuerzos y pérdida de control sobre los objetivos planteados.

   Por lo tanto, se requiere una aplicación de gestor de tareas que permita:
   - Centralizar la información de las actividades.
   - Facilitar la asignación de responsabilidades.
   - Priorizar y dar seguimiento a las tareas.
   - Mejorar la comunicación y productividad de los usuarios.

3. **Alcance del proyecto. (restricciones, criterios de aceptación del proyecto).**

   a. Registro e inicio de sesión. (Autenticación de Google).

- Funcionalidades: Creación y asignación de tareas, tablero Kanban.
- Visualización del progreso individual y grupal.
- Asignación de tareas a usuarios específicos.

   b. Filtros y búsqueda de tareas por estado, prioridad o usuario.

   c. Definición de título, descripción, fecha límite y prioridad.

   d. Actualización de estado (pendiente, en progreso, completada).

4. **Estructura del Desglose. (EDT)**

DT – Estructura de Desglose de Trabajo

1. Inicio del proyecto

1.1 Definición del alcance

1.2 Identificación de requerimientos

1.3 Elaboración del plan del proyecto

2. Análisis y diseño

2.1 Levantamiento de requerimientos funcionales y no funcionales

2.2 Diseño de la arquitectura del sistema

2.3 Diseño de la base de datos

2.4 Diseño de la interfaz de usuario (mockups, prototipos)

3. Desarrollo

3.1 Configuración del entorno de desarrollo

3.2 Implementación del backend (API REST / lógica de negocio)

3.3 Implementación del frontend (interfaz de usuario)

3.4 Integración frontend-backend

3.5 Implementación de autenticación y seguridad

3.6 Gestión de notificaciones y recordatorios

4. Pruebas

4.1 Pruebas unitarias

4.2 Pruebas de integración

4.3 Pruebas de usabilidad

4.4 Corrección de errores

5. Despliegue
5.1 Configuración del entorno en la nube (ej. GCP, AWS o Azure)
5.2 Implementación de CI/CD
5.3 Lanzamiento de la aplicación

6. Documentación y capacitación
6.1 Documentación técnica (arquitectura, APIs, base de datos)
6.2 Manual de usuario
6.3 Capacitación a usuarios finales

7. Cierre del proyecto
7.1 Evaluación de resultados
7.2 Retroalimentación de usuarios
7.3 Plan de mejoras futuras

5. Objetivos

General.

Desarrollar una aplicación web de gestión de tareas que permita a los usuarios organizar, priorizar y dar seguimiento a sus actividades de manera eficiente, fomentando la productividad individual y la colaboración en equipos de trabajo mediante una interfaz amigable, segura y accesible.

Objetivos Específicos.

a. Diseñar y desarrollar un módulo de gestión de tareas que permita crear, editar, eliminar y actualizar el estado de las actividades según su prioridad y fecha límite.
b. Desarrollar una interfaz de usuario intuitiva y responsiva, que garantice facilidad de uso en distintos dispositivos y entornos de trabajo.
c. Asegurar la escalabilidad y seguridad del sistema, implementando buenas prácticas de desarrollo, almacenamiento de datos.
d. Implementar un sistema de autenticación de usuarios que garantice el acceso seguro a la aplicación mediante registro, inicio de sesión con Google.

6. **Metodología ágil seleccionada (https://n9.cl/7z1xoa)**

Control de versiones y desarrollo colaborativo

a. GitHub / GitLab → Repositorios de código, control de versiones y gestión de ramas.
b. Gestión de proyectos y tareas

c. Trello → Tableros Kanban para organizar tareas por columnas (Pendiente, En progreso, Hecho).

Comunicación y colaboración

d. Microsoft Teams → Videollamadas, chat y almacenamiento colaborativo.

e. Google Workspace → Documentos, hojas de cálculo y calendarios compartidos.

7. **Justificación alcance corto, medio, largo), descripciones soluciones al problema, respuesta a los stakeholders.**

El desarrollo de una aplicación de gestión de tareas responde a la necesidad de organizar, priorizar y dar seguimiento a las actividades de usuarios individuales y equipos de trabajo, permitiendo aumentar la productividad, mejorar la comunicación y reducir la pérdida de información.

Actualmente, la falta de una herramienta centralizada genera retrasos, duplicidad de esfuerzos y baja eficiencia. Con este proyecto se busca ofrecer una solución accesible, escalable y adaptable a distintos contextos (académico, profesional o empresarial).

Corto plazo (primeros 6 meses)

a. Desarrollar una versión mínima viable (MVP) con funciones básicas: registro de usuarios, creación/edición de tareas, estados (pendiente, en progreso, completada).
b. Implementar un sistema seguro de autenticación.
c. Permitir la gestión individual de tareas.

Largo plazo (más de 12 meses)

d. Integrar la aplicación con calendarios externos (Google Calendar, Outlook).
e. Incorporar aplicación móvil nativa para Android y iOS.
f. Añadir inteligencia artificial para sugerencias de priorización y predicción de tiempos de entrega.
g. Expandir el sistema hacia entornos empresariales con gestión de proyectos avanzada.

8. Mapa de Stakeholders y clasificación de los mismos

Stakeholders

Stakeholder	**Rol**	**Nivel de Influencia**	**Nivel de Interés**	**Estrategia de Gestión**
Equipos de Desarrollo	Usuarios principales	Alto	Alto	Involucramiento continuo, feedback constante, priorización de sus necesidades
Líderes de Proyecto	Supervisores	Alto	Alto	Reuniones regulares, acceso prioritario a nuevas funcionalidades, soporte dedicado
Inversores	Financiamiento	Medio	Medio	Reportes periódicos de progreso, demostraciones de avances, gestión de expectativas
Departamento Legal	Cumplimiento normativo	Medio	Medio	Consultas regulares, revisiones de cumplimiento, documentación exhaustiva
Equipos Pequeños	Usuarios ocasionales	Bajo	Alto	Encuestas de satisfacción, canales de feedback, planes accesibles
Soporte Técnico	Atención al usuario	Bajo	Alto	Capacitación adecuada, documentación clara, herramientas de diagnóstico
Auditores Externos	Evaluación de seguridad	Bajo	Bajo	Mantener documentación actualizada, cumplir con estándares, transparencia limitada
Competidores	Otras empresas del sector	Bajo	Bajo	Monitoreo del mercado, Diferenciación clara, protección de propiedad intelectual

9. Matriz de riesgos.

ID	Riesgo	Probabilidad	Impacto	Nivel de Riesgo	Plan de Mitigación
R1	Retrasos en el desarrollo por cambios en los requerimientos	Media	Alto	Alto	Definir requerimientos claros desde el inicio y aplicar metodologías ágiles para manejar cambios.
R2	Fallas de seguridad en la autenticación de usuarios	Bajo	Alto	Medio-alto	Implementar cifrado de contraseñas, autenticación JWT y pruebas de seguridad.
R3	Baja adopción por parte de los usuarios finales	Media	Medio	Medio	Realizar pruebas de usabilidad y encuestas piloto para mejorar la experiencia del usuario.
R4	Problemas de disponibilidad del sistema en la nube	Baja	Alto	Medio	Usar servicios cloud con SLA garantizado y configurar monitoreo constante.
R5	Sobrecarga del sistema por aumento inesperado de usuarios	Medio	Alto	Alto	Diseñar una arquitectura escalable y realizar pruebas de carga.
R6	Pérdida de datos por fallos en la base de datos	Baja	Alto	Medio- Alto	Implementar backups automáticos y replicación de datos.
R7	Retrasos en la entrega por falta de experiencia técnica del equipo	Media	Medio	Medio	Capacitación del equipo y revisión continua con mentorías o code reviews.
R8	Costos mayores a lo presupuestado	Baja	Medio	Bajo-Medio	Establecer un control financiero con reportes periódicos.
R9	Errores en la integración frontend-backend	Media	Medio	Medio	Definir contratos claros en los endpoints de la API y realizar pruebas de integración continua.
R10	Dificultad para la adopción de metodologías ágiles en el equipo	Media	Medio	Medio	Capacitación en Scrum/Kanban y seguimiento con retrospectivas.

R11	Incompatibilidad entre navegadores o dispositivos	Alta	Medio	Medio-Alta	Realizar pruebas cross- browser y diseño responsive desde el inicio.
R12	Poca claridad en roles y responsabilidades dentro del equipo	Media	Alto	Alto	Definir un RACI (responsables, aprobadores, consultados, informados) y mantener comunicación continua.

10. Presupuesto.

Detalle por Fases (4 meses)

Fase	**Actividades**	**% presupuesto**	**Valor (COP)**
Mes 1 — Instalación & Configuración	SonarQube, Jenkins, Snyk, ZAP, setup infraestructura	20%	$12,800,000
Mes 2 – Integración	Pipelines CI/CD, pruebas automáticas, integración con repositorio	25%	$16,000,000
Mes 3 – Ejecución & Ajustes	Ejecución de análisis, pruebas de seguridad, tuning herramienta	30%	$19,200,000
Mes 4 — Validación & Entrega	Reportes finales, documentación, capacitación interna	25%	$16,000,000
Total		100%	**$64,000,000**

Distribución por Programador

a. $48,000,000 en 4 programadores durante 4 meses.
b. Equivale a $12,000,000 por programador en todo el proyecto.
c. Es decir, $3,000,000 mensuales aprox. por programador.

11. Levantamiento de información (herramientas y conclusiones)

1. SonarQube

Tipo: Análisis estático de código (SAST).
Características: Detecta bugs, vulnerabilidades, code smells y deuda técnica.
Ventajas: Open-source en su versión Community, soporta múltiples lenguajes, integración sencilla con pipelines CI/CD.
Desventajas: Configuración inicial compleja, requiere servidores con recursos adecuados.
Aplicación: Ideal para detectar problemas de seguridad y calidad

en el código fuente antes de compilar.

2. OWASP ZAP

Tipo: Análisis dinámico de seguridad (DAST).
Características: Escáner especializado en vulnerabilidades de aplicaciones web. Ventajas: Gratuito, mantenido por la comunidad OWASP, interfaz intuitiva.

Desventajas: Puede generar falsos positivos y requiere experiencia en ciberseguridad para interpretar resultados.

Aplicación: Detecta vulnerabilidades de seguridad OWASP Top 10 (XSS, SQLi, CSRF, etc.) durante pruebas dinámicas.

3. Snyk

Tipo: Análisis de dependencias y contenedores.
Características: Escaneo en tiempo real de librerías de terceros, imágenes Docker y configuraciones.
Ventajas: Base de datos de vulnerabilidades muy actualizada, integración directa con repositorios GitHub/GitLab.
Desventajas: El plan gratuito es limitado y los costos de la versión empresarial son altos.
Aplicación: Permite prevenir despliegues con librerías inseguras o configuraciones mal implementadas.

4. Jenkins

Tipo: Integración y despliegue continuo (CI/CD).
Características: Automatiza la compilación, ejecución de pruebas y despliegues en diferentes entornos.
Ventajas: Open-source, gran cantidad de plugin, adaptable a distintos proyectos.

Desventajas: Puede ser complejo de mantener en proyectos grandes y con muchas integraciones.

Aplicación: Orquesta herramientas como SonarQube, Snyk y Selenium dentro del pipeline de desarrollo.

5. Selenium

Tipo: Automatización de pruebas funcionales.
Características: Simula interacciones de usuario en navegadores, compatible con múltiples lenguajes.

Ventajas: Open-source, muy flexible, amplia comunidad de soporte.
Desventajas: Requiere mantenimiento constante de scripts, limitado para pruebas de Backend.
Aplicación: Automatiza pruebas de interfaz gráfica (UI) para validar la funcionalidad del sistema antes de producción.

Conclusiones del Levantamiento de Herramientas

El análisis realizado muestra que cada herramienta cumple un rol específico dentro del ciclo de vida del software:

SonarQube y Snyk permiten detectar vulnerabilidades de manera temprana en el código y en las dependencias.

OWASP ZAP aporta seguridad en la etapa de pruebas dinámicas, evaluando la aplicación en ejecución.

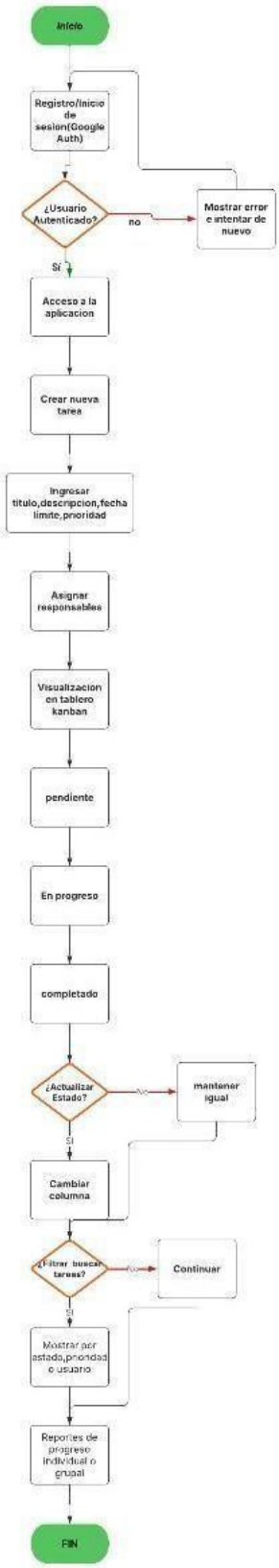
Jenkins y Selenium garantizan la automatización y la continuidad del proceso, evitando que las pruebas dependan únicamente de la intervención manual.

En conjunto, estas herramientas ofrecen una cobertura integral de calidad y seguridad, desde la codificación hasta el despliegue. Además, la mayoría cuenta con versiones gratuitas u open-source, lo que facilita su adopción en proyectos académicos y empresariales sin representar un alto costo.

12. Diagrama de flujo de solución selecciona.

Link diagrama de flujo (https://lucid.app/lucidchart/cd6052ec-e03e-4620-ad47-951309399339/edit?viewport_loc=277%2C443%2C1218%2C506%2C0_0&invitationId=inv_1987536e-fb51-4d4a-9965-267ab221240d)

Diagrama de flujo



Historias de Usuario en el tablero de la metodología Ágil seleccionada. Link (https://trello.com/invite/b/68d59cb7461e85b65a97f558/ATTI1329032e17cca957fadf00003137851317D2A48E/proyecto-de-software)

Autenticación y Acceso

1. Como usuario, quiero iniciar sesión en el sistema usando mi cuenta personal de Google para acceder de forma rápida y segura sin crear una nueva contraseña.
2. Como usuario, quiero cerrar sesión desde el sistema para proteger mi información cuando ya no necesite usar la aplicación.
3. Como administrador, quiero asegurar que solo los usuarios autenticados con Google puedan acceder al sistema para evitar accesos no autorizados.
4. Como administrador, quiero definir roles (administrador, miembro de equipo, observador) tras la autenticación para controlar qué acciones puede realizar cada usuario.
5. Como usuario, quiero que mi sesión permanezca activa mientras trabajo para no tener que iniciar sesión constantemente.
6. Como administrador, quiero poder revocar el acceso de un usuario autenticado para mantener la seguridad en caso de que alguien ya no deba usar la aplicación.

Gestión de tareas

7. Como usuario, quiero crear tareas con título, descripción, fecha límite y prioridad para organizar mis actividades de forma clara.
8. Como miembro de un equipo, quiero asignar tareas a usuarios específicos para distribuir responsabilidades y evitar confusión.
9. Como usuario, quiero cambiar el estado de mis tareas (pendiente, en progreso, completada) para reflejar el avance de mi trabajo.
10. Como usuario, quiero visualizar mis tareas en un tablero Kanban para tener una vista general de mi progreso.
11. Como miembro de un equipo, quiero ver el progreso grupal para evaluar el avance del proyecto en conjunto.
12. Como usuario, quiero filtrar y buscar tareas por estado, prioridad o usuario para encontrar fácilmente la información que necesito.
13. Como usuario, quiero comentar dentro de las tareas para mejorar la comunicación y resolver dudas sin depender de otras

herramientas.

14. Como usuario, quiero adjuntar documentos a las tareas para centralizar la información necesaria para cumplirlas.
15. Como usuario, quiero consultar el historial de cambios de una tarea para saber quién hizo modificaciones y en qué momento.

13. Definición de RQF-RQNF manejar un formato para la documentación.

Requisitos Funcionales. (RF)

ID	Requerimiento	Descripción
RF01	**Análisis de calidad del código**	El sistema deberá permitir realizar análisis de calidad del código fuente para detectar errores y malas prácticas.
RF02	**Detección de vulnerabilidades de seguridad**	El sistema deberá identificar posibles vulnerabilidades de seguridad en la aplicación.
RF03	**Ejecución de pruebas automáticas**	El sistema deberá ejecutar pruebas automáticas que validen el correcto funcionamiento de las principales funciones del software.
RF04	**Generación de reportes básicos**	El sistema deberá generar reportes básicos con los resultados de los análisis y pruebas realizados.
RF05	**Impedir paso a producción con fallas críticas**	El sistema deberá impedir el paso a producción de versiones que presenten fallas críticas.

Requerimientos No funcionales. (RNF)

ID	Requerimiento	Descripción
RNF01	**Facilidad de uso**	El sistema deberá ser fácil de utilizar por el equipo de desarrollo, con una interfaz clara y comprensible.
RNF02	**Integración con repositorios de control de versiones**	El sistema deberá integrarse de manera sencilla con repositorios de control de versiones como GitHub o GitLab.
RNF03	**Disponibilidad continua**	El sistema deberá estar disponible de forma continua durante el ciclo de desarrollo del proyecto.
RNF04	**Claridad en los reportes**	El sistema deberá presentar reportes con información clara y comprensible para los programadores.
RNF05	**Escalabilidad**	El sistema deberá ser escalable, de modo que pueda adaptarse a futuros proyectos sin grandes modificaciones.

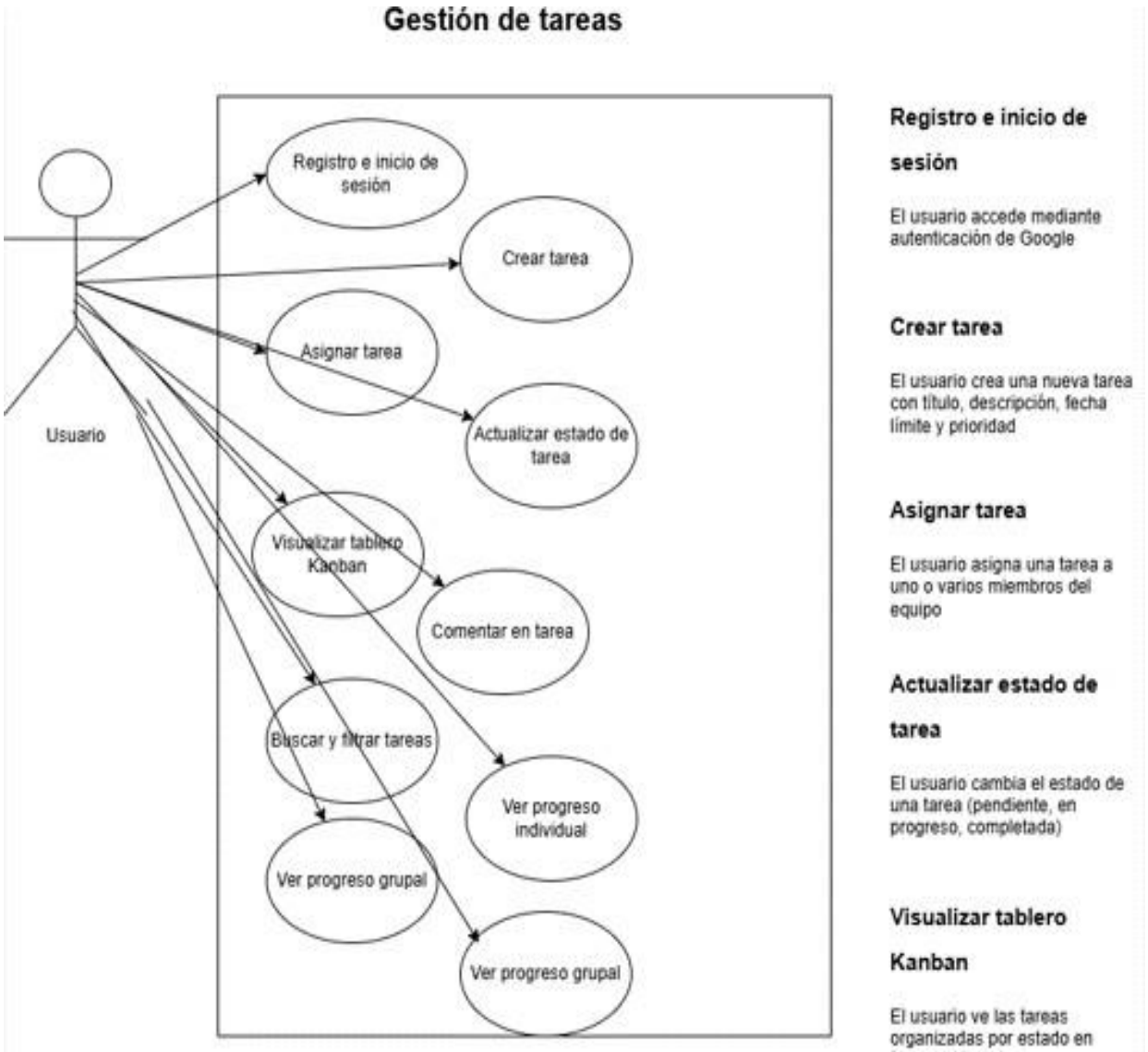
ACTIVIDAD 4 –DISEÑO DE PROTOTIPO

1. **Prototipo funcional**

Link https://skid-dairy-87515877.figma.site/

Link del video : https://laiberocol-my.sharepoint.com/:v:/g/personal/sleondur_estudiante_ibero_edu_co/EctpwqZa-zdIjf4M9GJESFkBNFhEaxXIhowFqEQJq4_zlw?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=uYlpOs

2. Casos de uso documentados



Registro e inicio de sesión

El usuario accede mediante autenticación de Google

Crear tarea

El usuario crea una nueva tarea con título, descripción, fecha límite y prioridad

Asignar tarea

El usuario asigna una tarea a uno o varios miembros del equipo

Actualizar estado de tarea

El usuario cambia el estado de una tarea (pendiente, en progreso, completada)

Visualizar tablero Kanban

El usuario ve las tareas organizadas por estado en formato Kanban

Comentar en tarea

El usuario deja comentarios en una tarea para facilitar la comunicación

Buscar y filtrar tareas

El usuario busca tareas por estado, prioridad o usuario asignado

Ver progreso individual

El usuario consulta su avance personal en las tareas asignadas

Ver progreso grupal

El usuario consulta el avance general del equipo en un proyecto

Registrar actividad

El sistema registra cada acción del usuario como parte del historial

ID Historia	*HU 1*	**Nombre**	*Registro e Inicio de Sesión*	**Peso**	*5*
Caso de Uso	*CU_001 Registro e Inicio de Sesión*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				

FUNCIONALIDAD	*Permitir que los usuarios accedan a la plataforma de gestión de tareas mediante autenticación con Google, garantizando seguridad, simplicidad y trazabilidad.*
RESULTADO	***Para** acceder a la plataforma de gestión de tareas de forma segura, rápida y sin necesidad de crear una cuenta manualmente. El usuario podrá autenticarse usando su cuenta de Google, lo que garantiza una experiencia fluida y confiable. Una vez autenticado, el sistema lo redirige al tablero principal, permitiéndole visualizar sus tareas y comenzar a interactuar con el sistema.*
Flujo Normal	*1. El usuario accede a la aplicación.* *2. Selecciona "Iniciar sesión con Google".* *3. El sistema redirige al servicio de autenticación de Google.* *4. Google valida las credenciales.* *5. Si es la primera vez, se crea el usuario en la base de datos.* *6. El sistema inicia sesión y redirige al tablero principal.*

CRITERIOS DE ACEPTACIÓN

#	Criterio	Condición (Dado)	Acción (Cuando)	Resultado (Entonces)
1	Acceso a autenticación externa	**El usuario accede a la aplicación**	Selecciona "Iniciar sesión con Google"	El sistema redirige al servicio de autenticación de Google
2	Validación de credenciales	**El usuario tiene una cuenta válida en Google**	Google valida las credenciales	El sistema permite el acceso al tablero principal
3	Registro automático de usuario	**El usuario no existe en la base de datos**	Google confirma identidad	El sistema crea el registro del usuario y lo inicia sesión

	nuevo			
4	Manejo de credenciales inválidas	**El usuario ingresa credenciales incorrectas**	Intenta iniciar sesión	El sistema muestra mensaje de error: “Credenciales invalidas”
5	Manejo de error de conexión	**El sistema no puede conectarse con Google**	El usuario intenta iniciar sesión	El sistema muestra mensaje de error: “Error de conexión”
6	Usuario bloqueado	**El usuario está marcado como inactivo o bloqueado**	Intenta iniciar sesión	El sistema muestra mensaje: “Acceso denegado”

Excepciones
• Error de conexión con Google → se muestra mensaje de “No se pudo conectar con el servicio”. • Credenciales inválidas → se muestra “Error de autenticación”. • Usuario bloqueado → se muestra “Acceso denegado”.

ID Historia	*HU 2*	**Nombre**	*Crear tarea*	**Peso**	*5*
Caso de Uso	*CU_002 Crear tarea*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				
FUNCIONALIDAD	*Permitir que los usuarios creen nuevas tareas dentro de un proyecto, definiendo sus atributos clave para facilitar la planificación, seguimiento y asignación.*				
RESULTADO (Para)	***Para** organizar el trabajo individual y grupal de forma estructurada, permitiendo que cada usuario registre tareas con información clave como título, descripción, fecha límite y prioridad. Esto facilita la planificación, el seguimiento del progreso y la asignación de responsabilidades dentro de un proyecto, mejorando la eficiencia y la colaboración del equipo.*				

Flujo Normal	*1. El usuario accede al módulo de tareas.* *2. Selecciona "Crear nueva tarea".* *3. Ingresa título, descripción, fecha límite y prioridad.* *4. El sistema valida los datos.* *5. La tarea se guarda en la base de datos y se muestra en el tablero.*

CRITERIOS DE ACEPTACIÓN

#	Criterio	Condición (Dado)	Acción (Cuando)	Resultado (Entonces)
1	Registro exitoso de tarea	El usuario accede al módulo de tareas	Completa todos los campos requeridos	El sistema guarda la tarea y la muestra en el tablero Kanban
2	Validación de campos obligatorios	El usuario deja campos vacíos	Intenta guardar la tarea	El sistema muestra mensaje de error indicando los campos faltantes
3	Validación de fecha límite	El usuario ingresa una fecha anterior a hoy	Intenta guardar la tarea	El sistema muestra mensaje de error: "Fecha inválida"
4	Manejo de error de base de datos	El sistema tiene un fallo al guardar	El usuario intenta crear la tarea	El sistema muestra mensaje: "No se pudo guardar la tarea"

Excepciones

• Campos obligatorios vacíos → se muestra “Complete todos los campos requeridos”. • Fecha límite inválida → se muestra “La fecha debe ser posterior a hoy”. • Error de base de datos → se muestra “No se pudo guardar la tarea”.

ID Historia	*HU3*	**Nombre**	*Asignar tarea*	**Peso**	*5*
Caso de Uso	*CU_003 Crear tarea*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				
FUNCIONALIDAD	*Permitir que los usuarios responsables distribuyan tareas entre los miembros del equipo, facilitando la colaboración, la trazabilidad y la gestión eficiente del trabajo.*				
RESULTADO (Para)	***Para** distribuir responsabilidades de manera clara y eficiente dentro del equipo, asegurando que cada tarea tenga uno o más responsables definidos. Esto permite mejorar la trazabilidad del trabajo, facilitar el seguimiento del progreso individual y grupal, y garantizar que ninguna tarea quede sin atender.*				
Flujo Normal	*1. El usuario abre una tarea existente.* *2. Selecciona “Asignar a usuario”.* *3. Elige uno o varios usuarios del equipo.* *4. El sistema guarda la asignación y notifica a los usuarios.*				
CRITERIOS DE ACEPTACIÓN					
#	**Criterio**	**Condición (Dado)**	**Acción (Cuando)**	**Resultado (Entonces)**	
1	Asignación exitosa	El usuario accede a una tarea existente	Selecciona uno o más usuarios para asignar	El sistema guarda la asignación y notifica a los usuarios	

2	Validación de usuario existente	El usuario selecciona un nombre no registrado	Intenta asignar la tarea	El sistema muestra mensaje: “Usuario no encontrado”
3	Prevención de duplicados	El usuario ya está asignado a la tarea	Intenta asignarlo nuevamente	El sistema muestra mensaje: “Este usuario ya está asignado”
4	Manejo de error de base de datos	El sistema falla al guardar la asignación	El usuario confirma la asignación	El sistema muestra mensaje: “No se pudo completar la asignación”

Excepciones
• Usuario no existe → se muestra “Usuario no encontrado”. • Usuario ya asignado → se muestra “Este usuario ya está asignado”. • Error al guardar → se muestra “No se pudo completar la asignación”.

ID Historia	*HU 4*	**Nombre**	*Actualizar estado de tarea*	**Peso**	*5*
Caso de Uso	*CU_004 Actualizar estado de tarea*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				
FUNCIONALIDAD	*Permitir que los usuarios asignados a una tarea actualicen su estado (por ejemplo: Pendiente, En progreso, Completada), reflejando el avance real del trabajo y facilitando el seguimiento del proyecto.*				
RESULTADO (Para)	***Para** reflejar el avance real del trabajo en cada tarea, permitiendo que los usuarios asignados actualicen su estado según el progreso alcanzado. Esto facilita el seguimiento por parte del equipo, mejora la visibilidad del proyecto y permite tomar decisiones oportunas basadas en el estado actual de las tareas.*				
Flujo Normal	*1. El usuario accede a una tarea.* *2. Selecciona el nuevo estado: Pendiente, En progreso, Completada.*				

	3. El sistema actualiza el estado y registra la actividad.

CRITERIOS DE ACEPTACIÓN				
#	**Criterio**	**Condición (Dado)**	**Acción (Cuando)**	**Resultado (Entonces)**
1	Cambio exitoso de estado	El usuario accede a una tarea asignada	Selecciona un nuevo estado válido	El sistema actualiza el estado y lo refleja en el tablero Kanban
2	Validación de estado permitido	El usuario selecciona un estado no reconocido	Intenta guardar el cambio	El sistema muestra mensaje: “Estado no válido”
3	Verificación de permisos	El usuario no tiene permisos sobre la tarea	Intenta cambiar el estado	El sistema muestra mensaje: “No tiene permisos para modificar”
4	Manejo de tarea bloqueada	La tarea está en estado cerrado o bloqueado	El usuario intenta modificarla	El sistema muestra mensaje: “Esta tarea no puede modificarse”
5	Registro de actividad	El usuario cambia el estado de una tarea	El sistema procesa el cambio	Se registra la acción en el historial de actividad
Excepciones				
• Estado no válido → se muestra “Estado no reconocido”. • Tarea bloqueada → se muestra “No se puede modificar esta tarea”. • Error de base de datos → se muestra “No se pudo actualizar el estado”.				

ID Historia	*HU5*	**Nombre**	*Visualizar tablero Kanban*	**Peso**	*5*
Caso de Uso	*CU_005 Visualizar tablero Kanban*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				

FUNCIONALIDAD	*Permitir que los usuarios visualicen todas las tareas organizadas por estado (Pendiente, En progreso, Completada, etc.) en un tablero Kanban interactivo, facilitando el seguimiento del trabajo y la gestión colaborativa.*
RESULTADO (Para)	***Para** obtener una vista clara y organizada del estado actual de todas las tareas del proyecto, agrupadas por su progreso. Esto permite al equipo identificar cuellos de botella, priorizar tareas pendientes y coordinar esfuerzos de forma visual e intuitiva, mejorando la eficiencia y la colaboración.*
Flujo Normal	*1. El usuario accede al módulo Kanban.* *2. El sistema carga todas las tareas organizadas por estado.* *3. El usuario puede arrastrar tareas entre columnas para cambiar su estado.*

CRITERIOS DE ACEPTACIÓN

#	Criterio	Condición (Dado)	Acción (Cuando)	Resultado (Entonces)
1	Carga exitosa del tablero	El usuario accede al módulo Kanban	El sistema consulta las tareas del proyecto	Se muestran las tareas agrupadas por estado en columnas
2	Interacción drag-and-drop	El usuario tiene permisos sobre una tarea	Arrastra la tarea a otra columna	El sistema actualiza el estado y refleja el cambio en tiempo real
3	Validación de acceso	El usuario no pertenece al proyecto	Intenta acceder al tablero	El sistema muestra mensaje: “No tiene acceso a este proyecto”
4	Manejo de error de carga	El sistema no puede recuperar las tareas	El usuario accede al tablero	El sistema muestra mensaje: “Error al cargar tareas”
5	Registro de actividad	El usuario mueve una tarea entre columnas	El sistema procesa el cambio	Se registra la acción en el historial de actividad

Excepciones

- Error al cargar tareas → se muestra “No se pudo cargar el tablero”.
- Usuario sin tareas → se muestra “No hay tareas asignadas”.

ID Historia	*HU 5*	**Nombre**	*Comentar en tarea*	**Peso**	*5*
Caso de Uso	*CU_005 Comentar en tarea*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				
FUNCIONALIDAD	*Permitir que los usuarios asignados a una tarea puedan agregar comentarios, observaciones o actualizaciones relevantes, fomentando la comunicación contextual y el trabajo colaborativo.*				
RESULTADO (Para)	***Para** facilitar la comunicación directa y contextual entre los miembros del equipo dentro de cada tarea, permitiendo registrar observaciones, avances, dudas o acuerdos sin salir del entorno de trabajo. Esto mejora la trazabilidad de decisiones, reduce la dispersión de información y fortalece la colaboración en tiempo real.*				
Flujo Normal	*1. El usuario abre una tarea.* *2. Escribe un comentario en el campo correspondiente.* *3. El sistema guarda el comentario y lo muestra en el historial.*				

CRITERIOS DE ACEPTACIÓN

#	Criterio	Condición (Dado)	Acción (Cuando)	Resultado (Entonces)
1	Comentario exitoso	El usuario está asignado a una tarea	Escribe un mensaje y lo envía	El sistema guarda el comentario con autor, fecha y hora
2	Validación de contenido	El campo de comentario está vacío	El usuario intenta enviarlo	El sistema muestra mensaje: “El comentario no puede estar vacío”
3	Visualización de comentarios	La tarea tiene comentarios previos	El usuario accede al detalle de la tarea	El sistema muestra los comentarios en orden cronológico
4	Mención de usuarios	El usuario incluye @nombre en el comentario	Envía el mensaje	El sistema notifica al usuario mencionado
5	Registro de	El usuario comenta en una tarea	El sistema procesa el comentario	Se registra la acción en el historial de actividad

	actividad			
Excepciones				
• Comentario vacío → se muestra “El comentario no puede estar vacío”. • Error al guardar → se muestra “No se pudo registrar el comentario”.				

ID Historia	*HU6*	**Nombre**	*Buscar y filtrar tareas*	**Peso**	*5*
Caso de Uso	*CU_006 Buscar y filtrar tareas*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				
FUNCIONALIDAD	*Permitir que los usuarios localicen rápidamente tareas específicas dentro de un proyecto mediante criterios de búsqueda y filtros, optimizando la navegación, el seguimiento y la toma de decisiones.*				
RESULTADO (Para)	***Para** localizar rápidamente tareas específicas dentro de un proyecto y enfocarse en lo más relevante según el contexto de trabajo. Esto permite ahorrar tiempo, mejorar la productividad y facilitar la toma de decisiones al mostrar únicamente las tareas que cumplen con los criterios definidos por el usuario.*				
Flujo Normal	*1. El usuario accede al buscador de tareas.* *2. Ingresa criterios: estado, prioridad, usuario.* *3. El sistema filtra y muestra los resultados.*				
CRITERIOS DE ACEPTACIÓN					
#	**Criterio**	**Condición (Dado)**	**Acción (Cuando)**	**Resultado (Entonces)**	
1	Búsqueda por palabra clave	El usuario escribe texto en el campo de búsqueda	Presiona “Buscar”	El sistema muestra tareas que coinciden con el texto ingresado	
2	Aplicación de filtros combinados	El usuario selecciona múltiples filtros (estado, prioridad, usuario)	Aplica los filtros	El sistema muestra solo las tareas que cumplen con todos los criterios	

3	Limpieza de filtros	El usuario ha aplicado filtros	Presiona “Limpiar filtros”	El sistema restablece la vista general de todas las tareas
4	Manejo de resultados vacíos	No existen tareas que cumplan con los criterios	Aplica búsqueda o filtros	El sistema muestra mensaje: “No se encontraron tareas”
5	Validación de acceso	El usuario no tiene permisos sobre ciertas tareas	Aplica filtros o búsqueda	El sistema no muestra tareas restringidas
Excepciones				
• Sin resultados → se muestra “No se encontraron tareas”. • Criterios inválidos → se muestra “Filtro no válido”.				

ID Historia	*HU 7*	**Nombre**	*Ver progreso individual*	**Peso**	*5*
Caso de Uso	*CU_007 Ver progreso individual*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				
FUNCIONALIDAD	*Permitir que cada usuario visualice el estado y avance de sus tareas asignadas, facilitando el seguimiento personal, la autoorganización y la mejora continua en la gestión del tiempo y responsabilidades.*				
RESULTADO (Para)	***Para** que cada usuario pueda monitorear el estado y avance de sus tareas asignadas, identificando pendientes, vencidas y completadas. Esto permite mejorar la autoorganización, priorizar actividades críticas y fomentar la responsabilidad individual dentro del equipo.*				
Flujo Normal	*1. El usuario accede a su perfil o panel de tareas.* *2. El sistema calcula el porcentaje de tareas completadas.* *3. Se muestra el progreso en formato gráfico o numérico.*				
CRITERIOS DE ACEPTACIÓN					
#	**Criterio**	**Condición (Dado)**	**Acción (Cuando)**	**Resultado (Entonces)**	

1	Visualización de tareas asignadas	El usuario tiene tareas en curso	Accede a la sección “Mi progreso”	El sistema muestra todas sus tareas agrupadas por estado
2	Indicadores de avance	El usuario tiene tareas en distintos estados	Accede al panel de progreso	El sistema muestra métricas como porcentaje completado y tareas vencidas
3	Aplicación de filtros	El usuario desea ver tareas por proyecto o prioridad	Aplica los filtros disponibles	El sistema muestra solo las tareas que cumplen con los criterios
4	Manejo de resultados vacíos	El usuario no tiene tareas asignadas	Accede a la sección de progreso	El sistema muestra mensaje: “No tienes tareas asignadas”
5	Actualización en tiempo real	El usuario cambia el estado de una tarea	Regresa al panel de progreso	El sistema actualiza automáticamente las métricas y agrupaciones
Excepciones				
	• Sin tareas asignadas → se muestra “No hay tareas para mostrar”. • Error de cálculo → se muestra “No se pudo calcular el progreso”.			

ID Historia	*HU 8*	**Nombre**	*Ver progreso grupal*	**Peso**	*5*
Caso de Uso	*CU_008 Ver progreso grupal*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				
FUNCIONALIDAD	*Permitir que los líderes de proyecto y miembros del equipo visualicen el avance colectivo de las tareas dentro de un proyecto, facilitando el monitoreo del rendimiento, la identificación de bloqueos y la toma de decisiones colaborativas.*				
RESULTADO (Para)	***Para** monitorear el avance colectivo del equipo dentro de un proyecto, identificando cuellos de botella, tareas vencidas y distribución de carga entre los miembros. Esto permite a los líderes y colaboradores tomar decisiones informadas, redistribuir tareas si es necesario y asegurar el cumplimiento de los objetivos del proyecto.*				

Flujo Normal	*1. El usuario accede al panel de proyecto.* *2. El sistema analiza el estado de todas las tareas del equipo.*
	3. Se muestra el avance general por usuario y por proyecto.

CRITERIOS DE ACEPTACIÓN

#	Criterio	Condición (Dado)	Acción (Cuando)	Resultado (Entonces)
1	Visualización consolidada	El usuario tiene acceso al proyecto	Accede al panel de progreso grupal	El sistema muestra tareas agrupadas por estado y por usuario
2	Indicadores de avance grupal	El proyecto tiene tareas en distintos estados	Accede al panel	El sistema muestra métricas como porcentaje completado y tareas vencidas
3	Distribución de carga	Hay múltiples usuarios con tareas asignadas	Accede al panel	El sistema muestra la cantidad de tareas por usuario
4	Aplicación de filtros	El usuario desea ver tareas por fecha, prioridad o responsable	Aplica los filtros	El sistema muestra solo las tareas que cumplen con los criterios
5	Actualización en tiempo real	Se modifica el estado de una tarea del proyecto	Regresa al panel de progreso grupal	El sistema actualiza automáticamente las métricas y agrupaciones

Excepciones

- Proyecto sin tareas → se muestra “No hay tareas en este proyecto”.
- Error de carga → se muestra “No se pudo mostrar el progreso grupal”.

ID Historia	*HU 9*	**Nombre**	*Registrar actividad*	**Peso**	*5*
Caso de Uso	*CU_009 Registrar actividad*				
HISTORIA					
ROL (Yo)	*Yo como Usuaro*				

FUNCIONALIDAD	*Capturar y almacenar automáticamente las acciones relevantes realizadas por los usuarios dentro del sistema, con el fin de mantener trazabilidad, facilitar auditorías internas y mejorar la transparencia del trabajo colaborativo.*
RESULTADO (Para)	***Para** mantener un historial detallado y confiable de las acciones realizadas por los usuarios en el sistema, facilitando la trazabilidad, auditoría y análisis del comportamiento dentro de los proyectos. Esto permite mejorar la transparencia, detectar errores o inconsistencias, y respaldar decisiones operativas basadas en evidencia.*
Flujo Normal	*1. El sistema detecta una acción del usuario (crear, comentar, cambiar estado).* *2. Registra la actividad con fecha, tipo y usuario.* *3. La actividad queda disponible para auditoría o historial.*

CRITERIOS DE ACEPTACIÓN

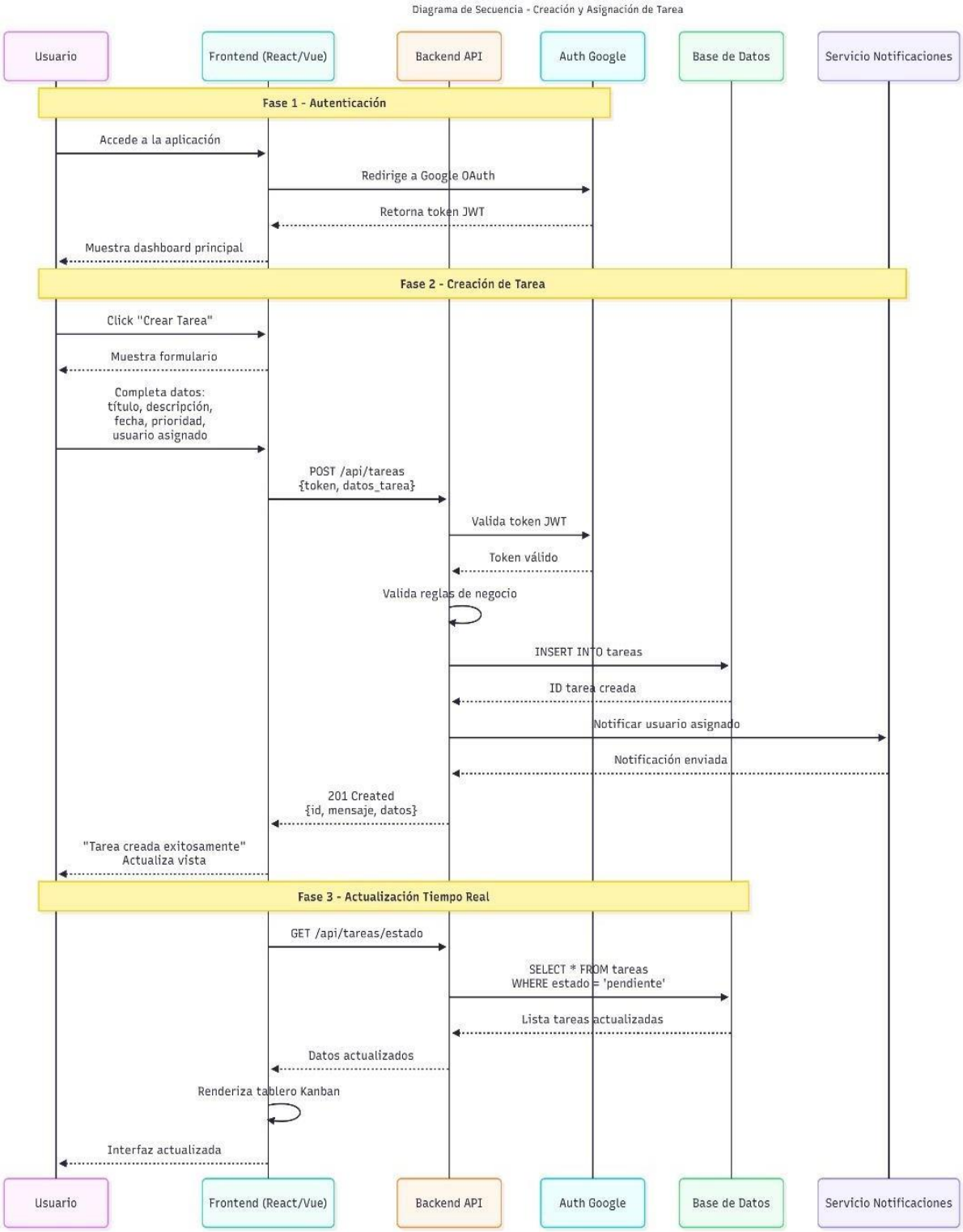
#	Criterio	Condición (Dado)	Acción (Cuando)	Resultado (Entonces)
1	Registro automático de acción	El usuario realiza una acción relevante (crear, editar, mover, comentar, etc.)	El sistema detecta el evento	Se genera un registro en la entidad ACTIVIDAD con fecha, autor y descripción
2	Visualización cronológica	Existen actividades registradas en una tarea	El usuario accede al historial de actividad	El sistema muestra los registros en orden cronológico
3	Protección de registros	El usuario accede a un registro de actividad	Intenta editar o eliminar el registro	El sistema bloquea la acción y muestra mensaje: “Registro no editable”
4	Filtro por usuario o tipo de acción	El usuario desea consultar actividades específicas	Aplica filtros en el panel de auditoría	El sistema muestra solo los registros que cumplen con los criterios
5	Manejo de errores	El sistema falla al registrar una actividad	Se realiza una acción relevante	

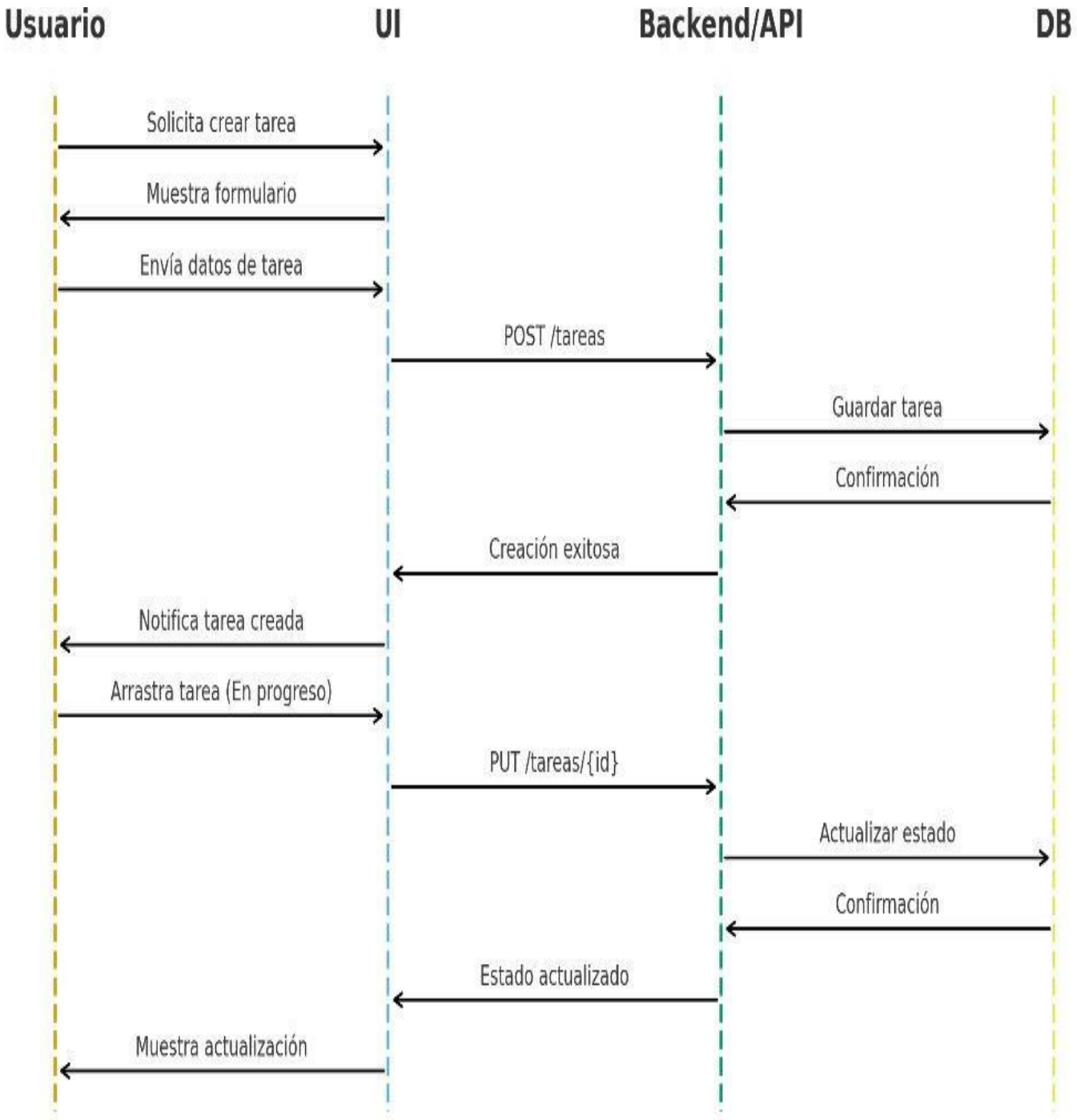
Excepciones

- Error de registro → se muestra “No se pudo guardar la actividad”.
- Acción no reconocida → se ignora o se muestra “Actividad no válida”.

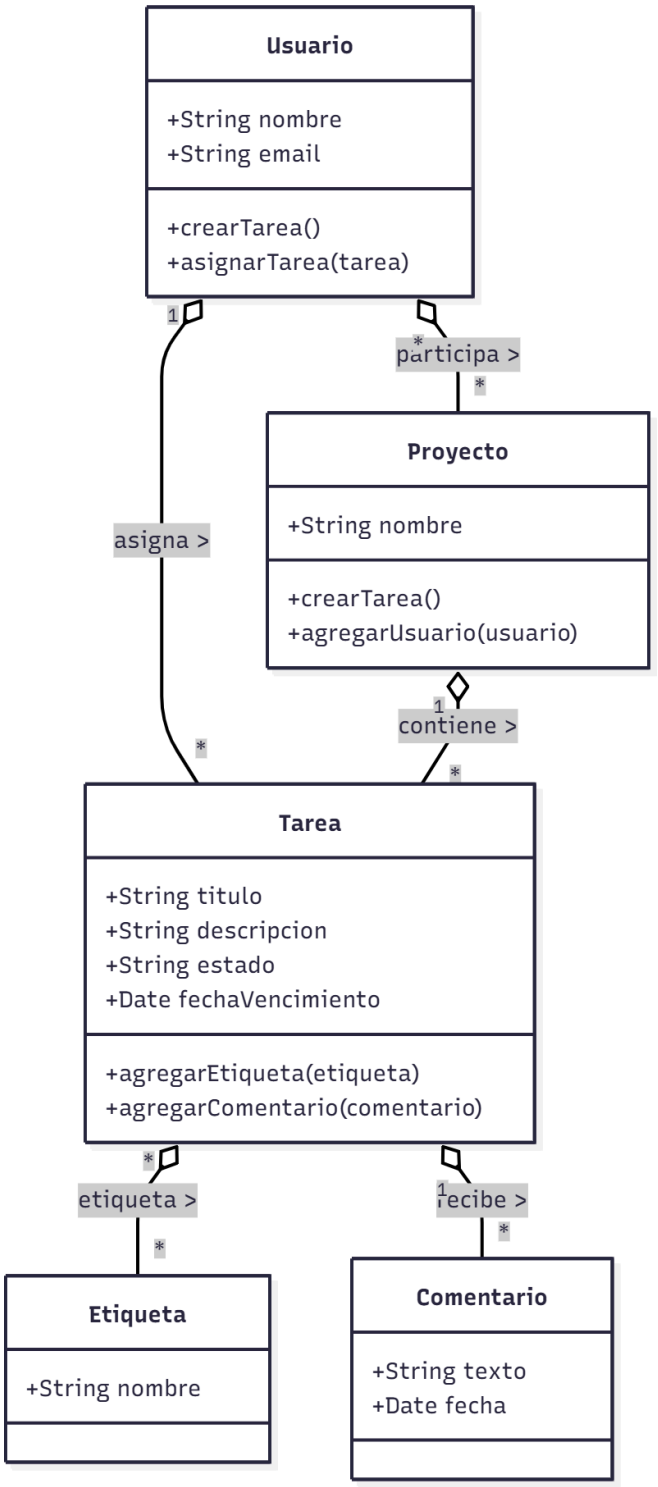
3. **Diagrama de secuencia**

Link diagrama:[https://www.mermaidchart.com/app/projects/dbac7b70-8f99-4f93- b7c6-af0d780a0319/diagrams/a906e96d-0ba1-4d02-a7fc-025c73393392/version/v0.1/edit](https://www.mermaidchart.com/app/projects/dbac7b70-8f99-4f93-b7c6-af0d780a0319/diagrams/a906e96d-0ba1-4d02-a7fc-025c73393392/version/v0.1/edit)

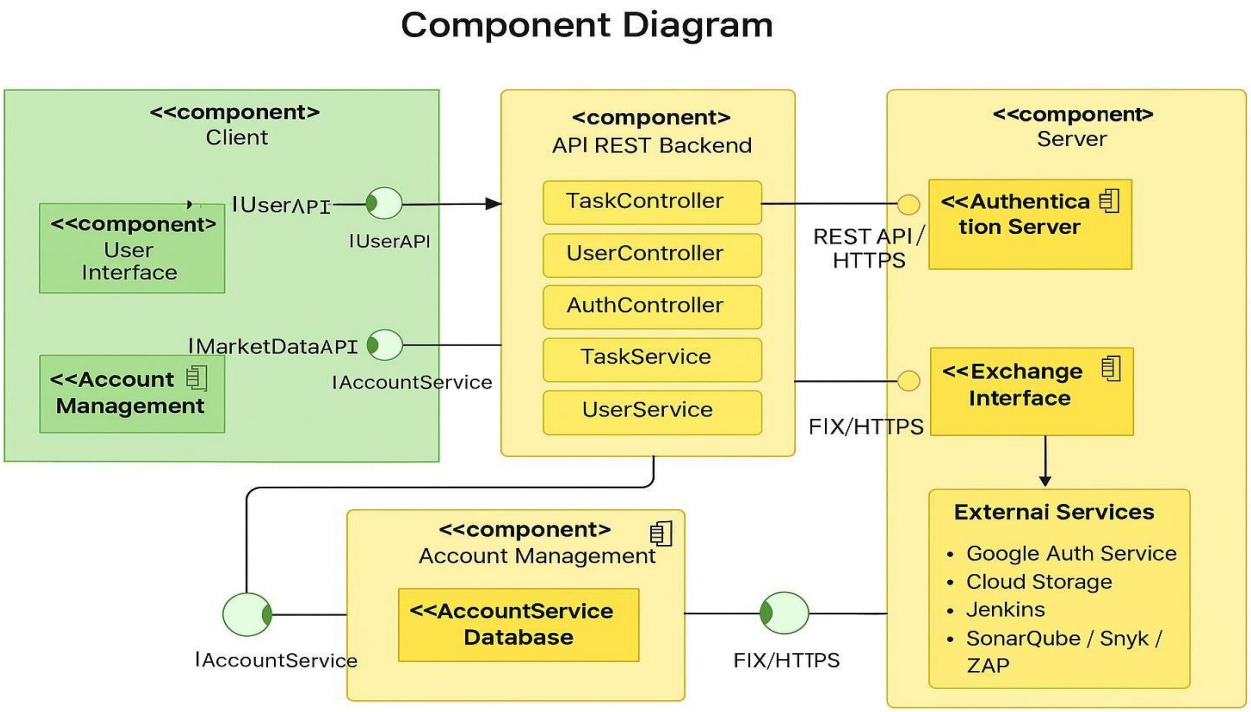



Usuario
UI
Backend/API
DB
Solicita crear tarea
Muestra formulario
Envía datos de tarea
POST /tareas
Guardar tarea
Confirmación
Creación exitosa
Notifica tarea creada
Arrastra tarea (En progreso)
PUT /tareas/{id}
Actualizar estado
Confirmación
Estado actualizado
Muestra actualización

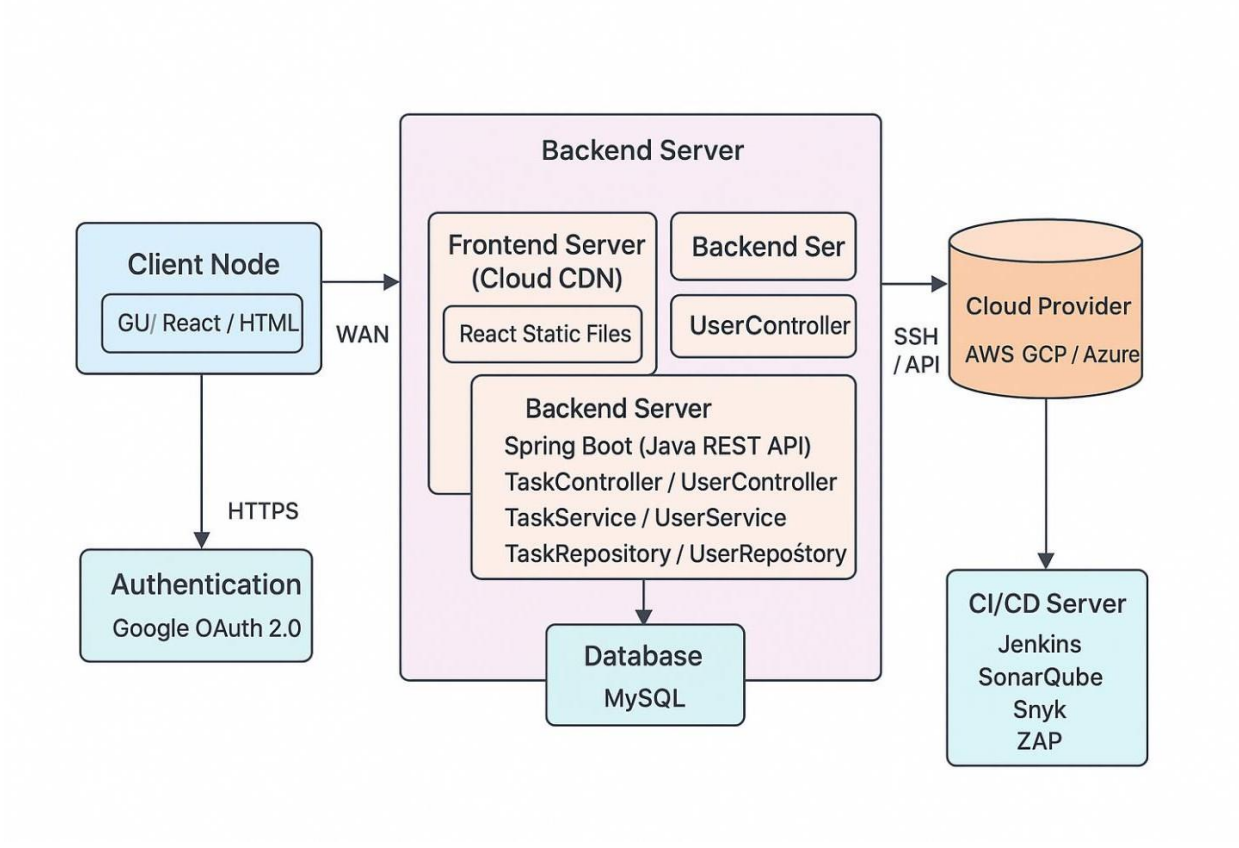
4. Diagrama de clases



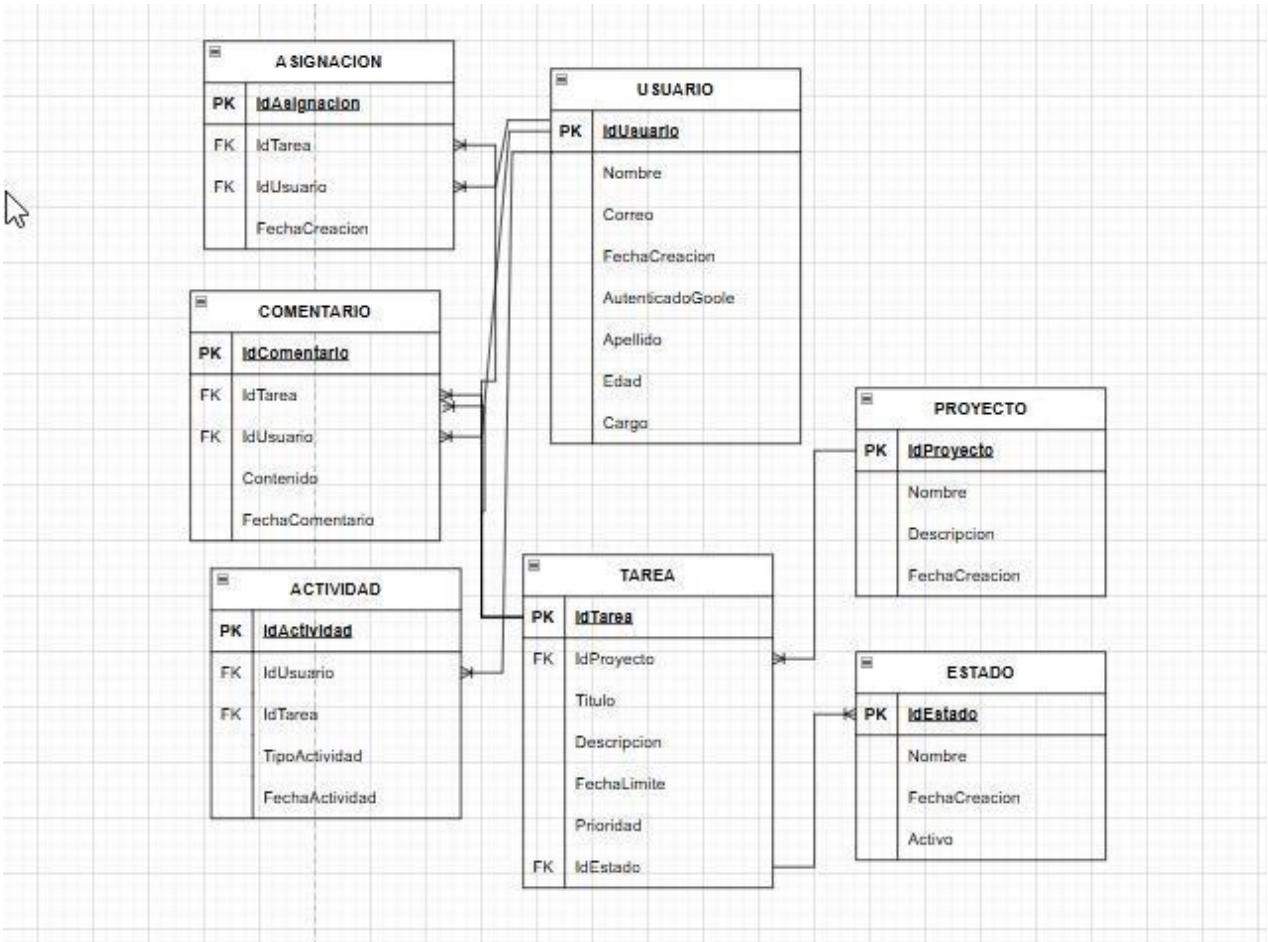
5. **Diagrama de componentes**



6. **Diagrama de despliegue**

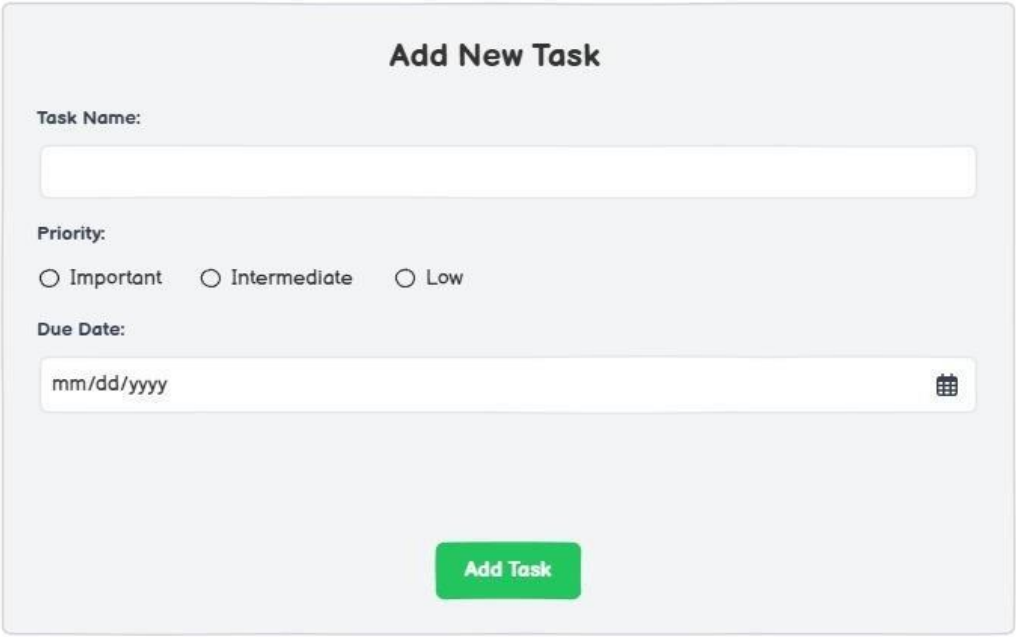


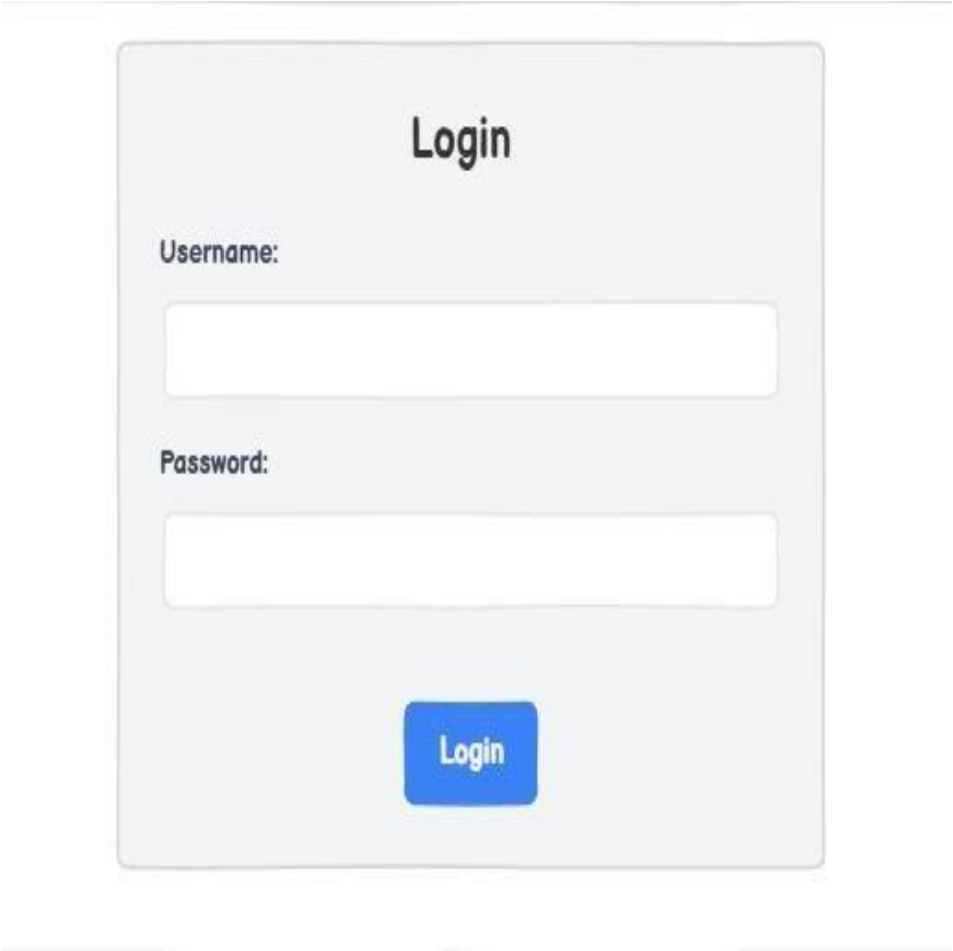
7. Modelo de base de datos



Prototipos de interfaz:

Baja fidelidad: https://balsamiq.cloud/smawjo4/pnqqh5a





Progreso de Tareas

Visualización del rendimiento individual y del equipo

Exportar Informe

Proyecto: Proyecto Alpha Periodo: Este Mes

Progreso General del Equipo

Tareas Completadas

28/40 (70%)

4 tareas pendientes, 8 en curso

Distribución de Tareas por Estado

Completadas	28
En Curso	8
Pendientes	4
Bloqueadas	0

Completadas · En Curso · Pendientes · Bloqueadas

Progreso por Categoría de Tarea

- Desarrollo (45%)
- Diseño (25%)
- Pruebas (20%)
- Documentación (10%)

Progreso Individual del Equipo

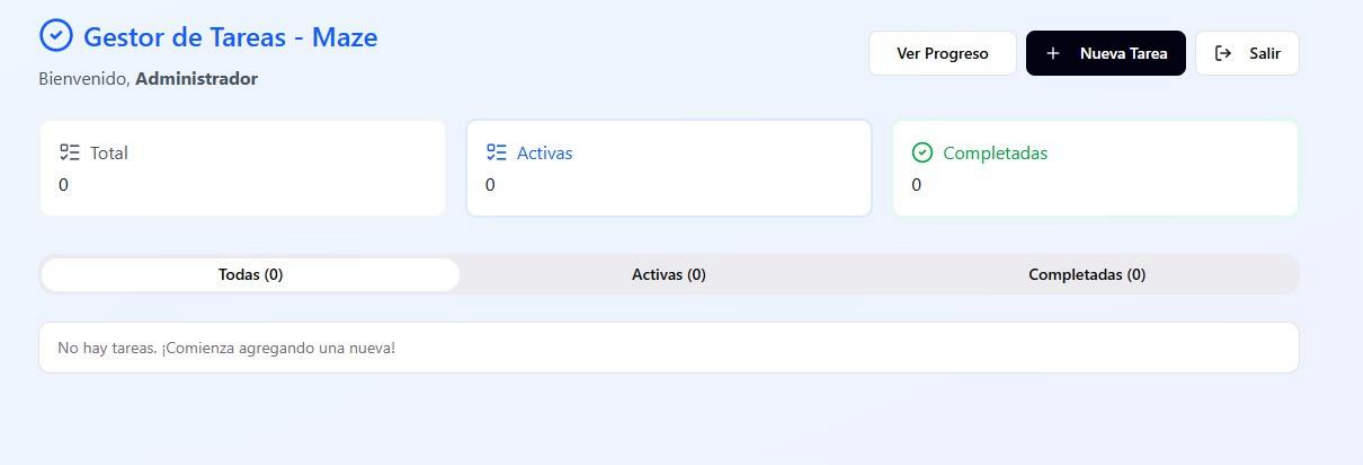
Juan Pérez
Desarrollador Frontend
5/6 tareas completadas — 85%

María García
Diseñadora UX/UI
3/5 tareas completadas — 60%

Pedro López
Desarrollador Backend
6/8 tareas completadas — 75%

Ana Rodríguez
QA Tester
9/10 tareas completadas — 90%

Alta fidelidad: https://skid-dairy-87515877.figma.site/



Progreso del Equipo

Visualiza el avance individual y grupal

Mi Progreso

0 de 0 tareas completadas

0% completado

Progreso Grupal

0 de 0 tareas completadas por el equipo

Total	Activas	Completadas
0	0	0

Progreso por Persona

Administrador (Tú) 0%
@admin 0/0 tareas

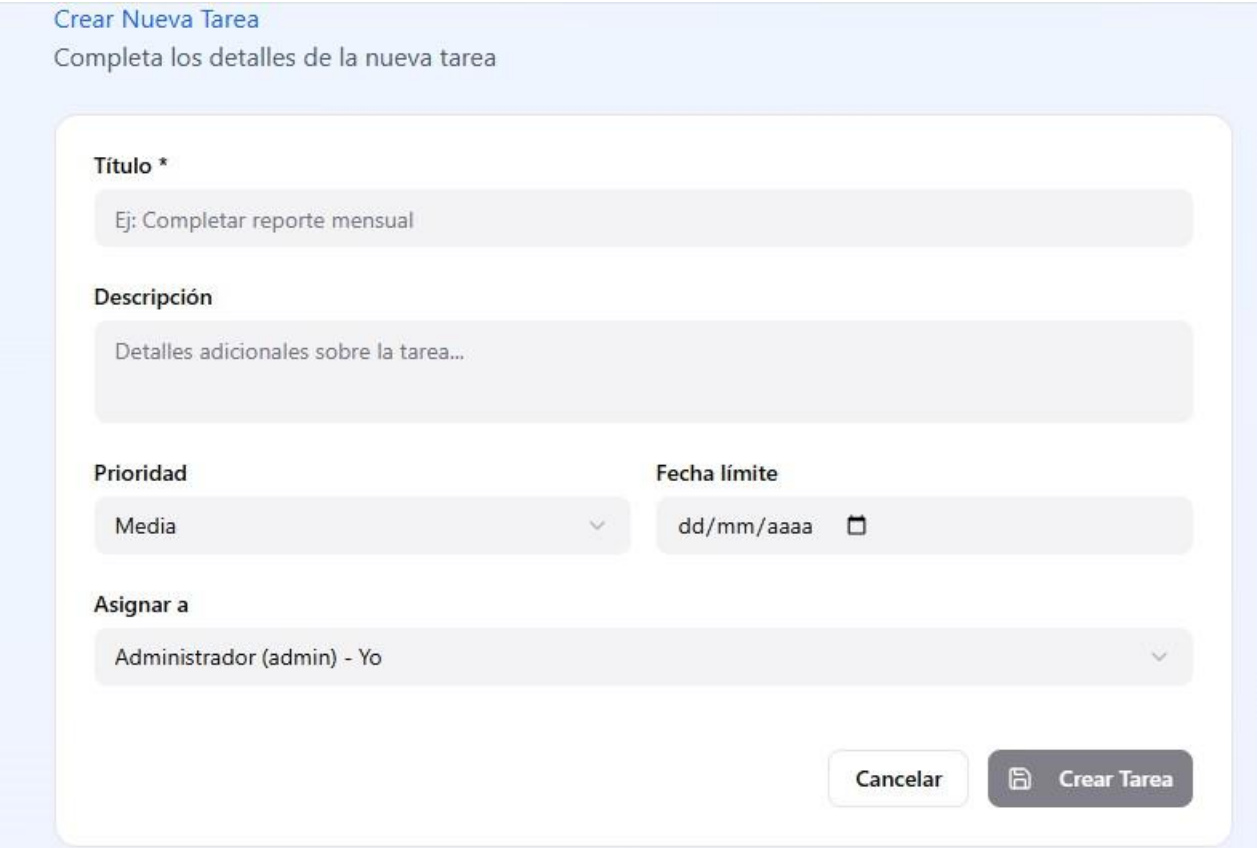
Total	Activas	Completadas
0	0	0

María García 0%
@maria 0/0 tareas

Total	Activas	Completadas
0	0	0

Juan Pérez 0%
@juan 0/0 tareas

Total	Activas	Completadas
0	0	0



8. **Pruebas de Usabilidad:**

LINK MAZE: https://t.maze.co/463811243

Gesto de tareas Versión del prototipo:1.0
Fecha de la prueba 9/11/2025 Santiago, Jeison, Yolanda y Jhon
Herramientas usadas (Figma, Maze)
Objetivos de la prueba: probar el funcionamiento del agendamiento de tareas y designación
Número de participantes:5
Escenarios: Crear, editar, completas tareas y salir del sistema
Resultados: El tiempo promedio es de 50 segundos, la tasa de éxito fue del 60%, la tasa de fallo fue del 40%,
Observaciones: Añadir nuevas funcionalidades en un futuro
Patrones y Arquitectura de Software:

1. Seleccionar e incluir los patrones arquitectónicos y de diseño aplicados al proyecto.

Patrones Arquitectónicos Aplicados

1. Arquitectura en Capas (Layered Architecture)

Descripción:
Divide el sistema en capas lógicas con responsabilidades bien definidas.
Aplicación en el proyecto:

- Capa de presentación (Frontend): Interfaz desarrollada en React o similar, conectada con la API REST.
- Capa de negocio (Backend): Lógica de negocio, control de usuarios, tareas, y roles.
- Capa de datos (Persistencia): Base de datos (por ejemplo, MySQL o Firebase) donde se almacenan usuarios, tareas y estados.

Ventajas:

- Facilita el mantenimiento.
- Permite el reemplazo o actualización de componentes sin afectar todo el sistema.

2. Arquitectura Cliente–
Servidor Descripción:
Modelo clásico donde el cliente realiza solicitudes a un servidor central.
Aplicación en el proyecto:

- Cliente: Navegador del usuario ejecutando el frontend (React, HTML, CSS, JS).
- Servidor: API REST (Node.js/Express) que procesa las solicitudes y accede a la base de datos.

Ventajas:

- Comunicación clara entre cliente y servidor.
- Escalabilidad horizontal posible mediante microservicios.

3. Arquitectura
RESTful
Descripción:
Estilo arquitectónico que estructura las API basadas en recursos y métodos HTTP.
Aplicación en el proyecto:

- Endpoints para tareas, usuarios y autenticación (GET, POST, PUT, DELETE).
- JSON como formato de intercambio de datos.

Ventajas:

- Interoperabilidad entre sistemas.
- Reutilización y claridad en la API.

4. Patrón MVC (Modelo–Vista–
Controlador) Descripción:
Separa la lógica de negocio, la interfaz de usuario y el control del flujo.
Aplicación en el proyecto:

- Modelo: Entidades como Usuario, Tarea, Rol.
- Vista: Interfaz de usuario (frontend).

- Controlador: API REST que recibe las peticiones y ejecuta la lógica necesaria.

Ventajas:

- Facilita la mantenibilidad y las pruebas.
- Permite trabajar en paralelo en diferentes capas.

5. Arquitectura basada en Componentes Descripción:

El frontend se compone de componentes reutilizables e independientes.

Aplicación en el proyecto:

- Cada sección (tablero Kanban, lista de tareas, formulario de creación, login) es un componente React.

Ventajas:

- Reutilización y modularidad.
- Mantenimiento sencillo y escalabilidad en la interfaz.

Patrones de Diseño Aplicados

1. Patrón Singleton Uso:

Para la configuración global (por ejemplo, conexión a la base de datos o manejo de tokens de autenticación).

Ejemplo:

Un único objeto encargado de manejar la conexión con Firebase o con el servicio OAuth de Google.

2. Patrón Observer (Observador) Uso:

Para actualizar la interfaz en tiempo real cuando cambian las tareas o estados.

Ejemplo:

Cuando un usuario completa una tarea, los demás usuarios ven el cambio inmediatamente (puede implementarse con WebSockets o servicios en la nube como Firebase Realtime Database).

3. Patrón Factory Method Uso:

Para crear objetos (por ejemplo, tareas, usuarios o notificaciones) sin especificar la clase exacta.

Ejemplo:

Una fábrica de tareas que genera instancias según el tipo (pendiente, en progreso, completada).

4. Patrón DAO (Data Access Object) Uso:

Para abstraer la lógica de acceso a la base de datos.

Ejemplo:

Clases o módulos que manejan las consultas a la base de datos (crear, leer, actualizar, eliminar tareas).

5. Patrón Strategy

Uso:

Para manejar distintas estrategias de filtrado o búsqueda de tareas (por prioridad, estado, usuario).

Ejemplo:

Cada tipo de filtro se implementa como una estrategia intercambiable.

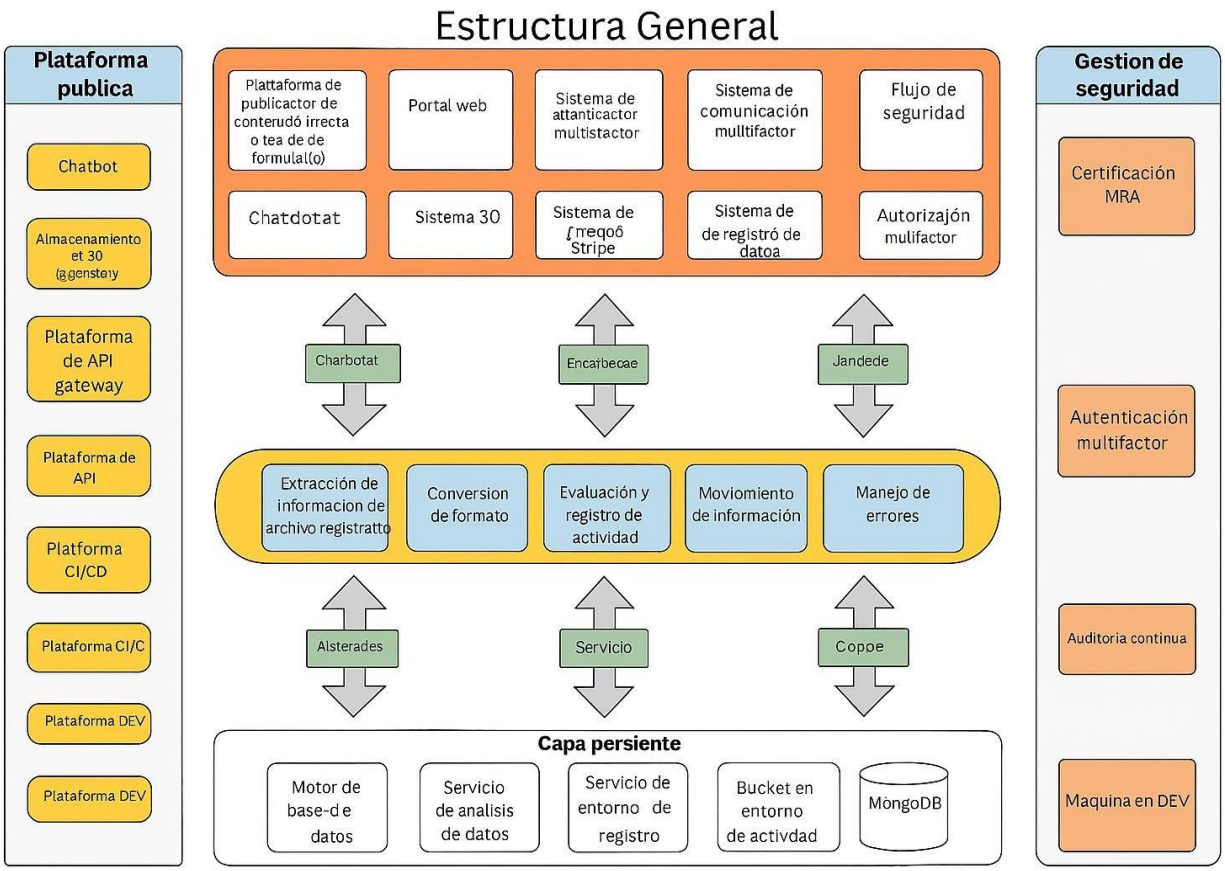
6. Patrón Dependency Injection Uso:

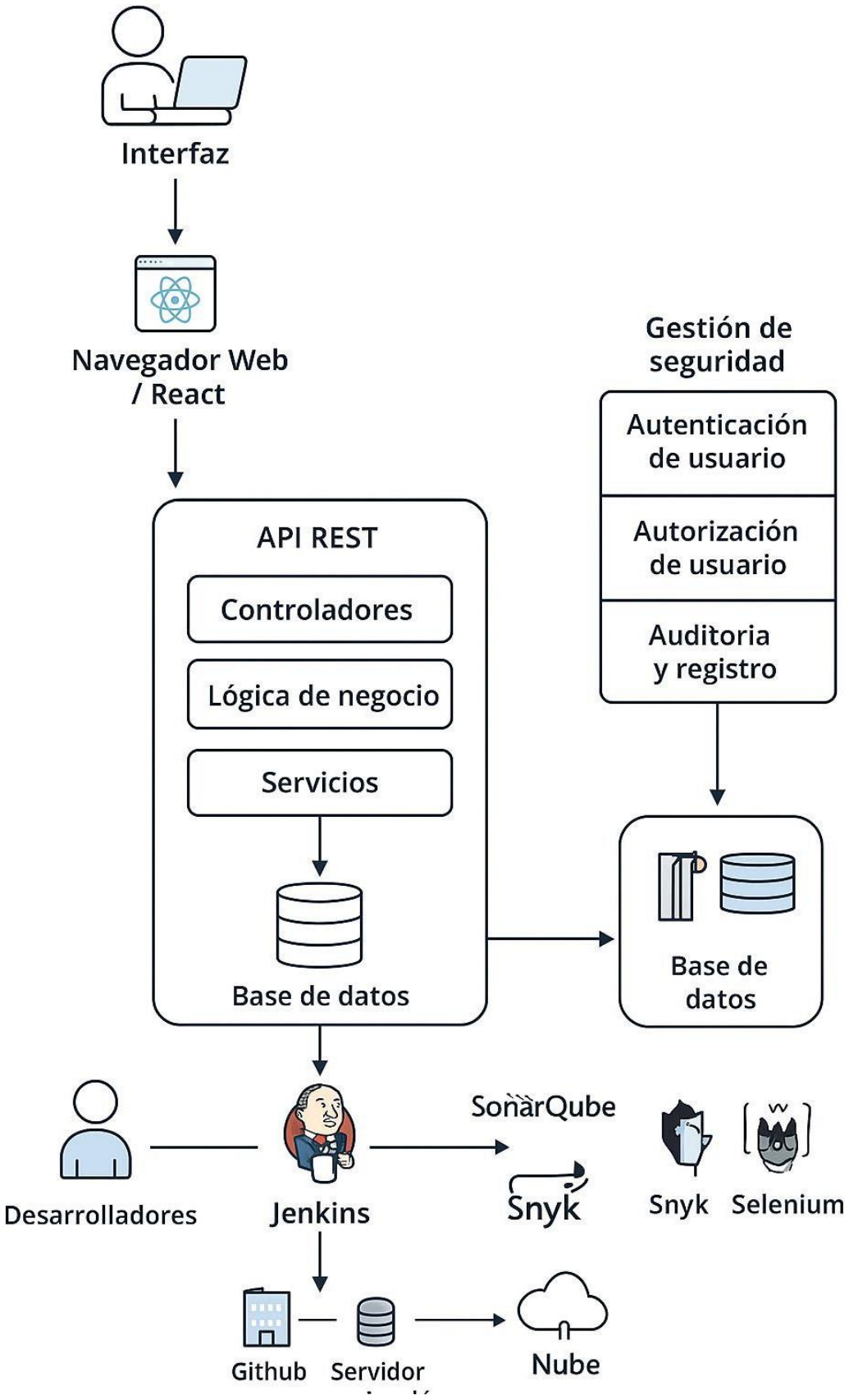
Para desacoplar las dependencias entre módulos (por ejemplo, inyectar servicios como repositorios o controladores).

Ejemplo:

Los controladores reciben el servicio de tareas como dependencia, facilitando pruebas unitarias.

Tipo	**Patrón**	**Aplicación en el proyecto**	**Beneficio principal**
Arquitectónico	En capas	Separación presentación/negocio/datos	Mantenimiento fácil
Arquitectónico	Cliente–Servidor	Comunicación entre frontend y backend	Escalabilidad
Arquitectónico	RESTful	API estructurada por recursos	Interoperabilidad
Arquitectónico	MVC	División clara de responsabilidades	Reutilización de código
Arquitectónico	Basado en componentes	UI modular (React)	Reutilización visual
Diseño	Singleton	Conexión base de datos	Control único de instacias
Diseño	Observer	Actualización en tiempo real	Sincronización de datos
Diseño	Factory	Creación flexible de objetos	Extensibilidad
Diseño	DAO	Acceso a datos desacoplado	Mantenibilidad
Diseño	Strategy	Filtros de tareas dinámicos	Flexibilidad
Diseño	Dependency Injection	Inyección de servicios	Pruebas y modularidad


Estructura General
Plataforma publica
Chatbot
Almacenamiento et 30 (agenstony
Plataforma de API gateway
Plataforma de API
Platforma CI/CD
Plataforma CI/C
Plataforma DEV
Plataforma DEV
Plattaforma de publicactor de conterudó irrecta o tea de de formulal(o)
Portal web
Sistema de attanticactor multistactor
Sistema de comunicación multifactor
Flujo de seguridad
Chatdotat
Sistema 30
Sistema de ʃ meqoô Stripe
Sistema de de registró de datoa
Autorizajón mulifactor
Charbotat
Encarbecae
Jandede
Extracción de informacion de archivo registratto
Conversion de formato
Evaluación y registro de actividad
Moviomiento de información
Manejo de errores
Alsterades
Servicio
Copoe
Capa persiente
Motor de base-d e datos
Servicio de analisis de datos
Servicio de entorno de registro
Bucket en entorno de activdad
MongoDB
Gestion de seguridad
Certificación MRA
Autenticación multifactor
Auditoria continua
Maquina en DEV


Interfaz
Navegador Web / React
Gestión de seguridad
Autenticación de usuario
Autorización de usuario
Auditoria y registro
API REST
Controladores
Lógica de negocio
Servicios
Base de datos
Base de datos
Desarrolladores
Jenkins
SonarQube
Snyk
Snyk
Selenium
Github
Servidor
Nube

Documentación de la Arquitectura de Software

1. Justificación de la Arquitectura de Software

La arquitectura seleccionada para el proyecto “Aplicación Web de Gestión de Tareas” se fundamenta en el patrón arquitectónico por Capas (Client–Server / N-Tier), ya que ofrece una estructura modular, escalable y segura que se adapta perfectamente a los requerimientos funcionales y no funcionales definidos en el documento de proyecto.
Este patrón divide la aplicación en tres capas principales:

- Presentación (Frontend)
- Lógica de Negocio (Backend / API REST)
- Acceso a Datos (Base de Datos y Repositorios)

La comunicación entre estas capas se realiza de manera jerárquica y controlada, lo que permite mantener una separación clara de responsabilidades y mejorar la mantenibilidad del sistema.

Justificación Técnica y Estratégica

Aspecto	Justificación	Contribución al Proyecto
Separación de responsabilidades	Cada capa se encarga de un conjunto específico de tareas: la interfaz presenta la información, la lógica gestiona las reglas del negocio y la base de datos persiste los datos.	Facilita el desarrollo colaborativo, mantenimiento y escalabilidad del sistema.
Escalabilidad y rendimiento	Permite escalar horizontal o verticalmente cada capa según la carga de trabajo, lo que mejora el rendimiento ante un aumento de usuarios.	Cumple con el Requisito No Funcional RNF05 y mitiga el Riesgo R5: Sobrecarga del sistema por aumento inesperado de usuarios.
Seguridad	La capa intermedia (Backend) actúa como un punto de control que filtra solicitudes, aplica autenticación y valida accesos antes de llegar a la base de datos.	Cumple con el objetivo de implementar un sistema seguro de autenticación mediante Google OAuth y mitiga el Riesgo R2: Fallas de seguridad en la autenticación de usuarios.

Mantenibilidad y flexibilidad	Los cambios en la interfaz o en la base de datos no afectan la lógica del negocio, lo que agiliza las actualizaciones y el desarrollo de nuevas funcionalidades.	Mitiga el Riesgo R9: Errores en la integración Frontend–Backend y garantiza una evolución ágil del software.

2. **Implementación de la Arquitectura por Capas**

Capa	Componentes Principales	Implementación Específica en el Proyecto
1. Capa de Presentación (Frontend)	Interfaz web desarrollada con React.js, HTML, CSS y JavaScript.	Se encarga de la visualización de tareas, el tablero Kanban y la interacción con el usuario. Consume los servicios del Backend mediante peticiones API REST. Implementa autenticación con Google OAuth para el inicio de sesión seguro.
2. Capa de Lógica de Negocio (Backend / API REST)	Servidor implementado en Node.js con Express, organizado en módulos y controladores.	Procesa la lógica del negocio: creación, edición, asignación y seguimiento de tareas, manejo de usuarios y roles. Gestiona las validaciones y seguridad mediante JWT y control de roles. Expuesto como una API REST para interoperabilidad con otros clientes (web o móviles).
3. Capa de Acceso a Datos (Persistencia)	Base de datos MySQL (o Firebase para entornos en la nube). ORM / DAO para el manejo de entidades.	Gestiona las operaciones CRUD (crear, leer, actualizar y eliminar tareas y usuarios). Se implementan backups automáticos y replicación para mitigar el Riesgo R6: Pérdida de datos por fallos en la base de datos.

3. **Flujo de Desarrollo e Integración Continua (CI/CD)**

Para garantizar la calidad del software, se implementó un pipeline automatizado de Integración y Despliegue Continuo (CI/CD) que abarca análisis de código, pruebas, seguridad y despliegue en la nube.

Herramientas y Función en el Proyecto

Herramienta	Tipo	Función dentro del Proyecto
Jenkins	Integración y Despliegue Continuo	Automatiza la compilación del proyecto, ejecución de pruebas y despliegue en la nube (AWS/GCP).
SonarQube	Análisis Estático de Código (SAST)	Evalúa la calidad del código, detecta errores, malas prácticas y vulnerabilidades.
Snyk	Análisis de Dependencias	Escanea librerías externas y evita el uso de dependencias vulnerables.
OWASP ZAP	Análisis Dinámico (DAST)	Simula ataques para detectar vulnerabilidades en tiempo de ejecución (OWASP Top 10).
Selenium	Pruebas Funcionales Automáticas	Valida el funcionamiento correcto de la interfaz (UI) mediante pruebas automáticas.
GitHub	Repositorio y Control de Versiones	Centraliza el código fuente, documentación y configuración de CI/CD.

4. **Elementos Documentados en el Repositorio del Grupo**

Toda la información técnica del sistema se encuentra disponible y actualizada en el repositorio del grupo, dentro de la carpeta /docs/, donde se documentan las decisiones arquitectónicas, el pipeline y los diagramas de diseño.

Repositorio del Código
Fuente:https://github.com/jrodr337ibero/PROYECTO_SOFTWARE.git
Documentos y Recursos Incluidos

Archivo / Enlace	Contenido Documentado
/docs/Arquitectura_Software.pdf	Descripción técnica de las capas, justificación y diagramas de arquitectura.
Archivo / Enlace	Contenido Documentado
/docs/Pipeline_CI_CD.md	Configuración y flujo del proceso de integración continua con Jenkins, SonarQube, Snyk y Selenium.
/docs/Manual_Técnico.md	Endpoints de la API REST, estructura de datos y reglas de negocio.
/README.md	Descripción general del proyecto, objetivos y enlaces a herramientas.

5. **Documentación Complementaria en el Documento del Proyecto**

Los siguientes recursos se incluyen como soporte de la arquitectura y del diseño implementado:

- ***Diagrama de Flujo (Lucidchart):https://lucid.app/lucidchart/cd6052ec-e03e-4620-ad47-951309399339/edit***
- ***Diagrama de Secuencia (Mermaid): https://www.mermaidchart.com/app/projects/dbac7b70-8f99-4f93-b7c6-af0d780a0319/diagrams/a906e96d-0ba1-4d02-a7fc-025c73393392/version/v0.1/edit***
- ***Prototipo de Interfaz de Usuario (Figma): https://skid-dairy-87515877.figma.site/***
- ***Tablero de Metodología Ágil (Trello): https://trello.com/invite/b/68d59cb7461e85b65a97f558/ATTI1329032e17cca957fadf 00003137851317D2A48E/proyecto-de-software***

6. **Conclusión**

La arquitectura por capas implementada en este proyecto garantiza un desarrollo ordenado, escalable y seguro, la integración del pipeline de CI/CD con herramientas de análisis estático, dinámico y pruebas funcionales automatizadas asegura la calidad y estabilidad del software durante todas las fases del ciclo de vida.
Esta documentación permite mantener la trazabilidad del proyecto, facilita la comprensión técnica del sistema y constituye la base para futuras fases de expansión, como la creación de aplicaciones móviles o la incorporación de inteligencia artificial para priorización de tareas.

Actividad 6 Entrega final del proyecto.

LINK DEL VIDEO :https://laiberocol-my.sharepoint.com/:v:/g/personal/sleondur_estudiante_ibero_edu_co/IQBsmC7xG1EQTYmMC_YwYmG6ATPNXciNy09jex1KzGTtxNc?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=Z6XFAo

Links de repositorios
Link fronted: https://github.com/JhonnSr10/GESTOR-TAREA-FRONT.git
Link backend: https://github.com/JhonnSr10/GESTOR-TAREAS-BACKEND.git

Información general

Campo	Descripción
Proyecto	Sistema Gestor de Tareas - Backend
Versión	1.0
Fecha	2024
Responsable	Equipo QA

Pruebas unitarias (backend)

Módulo de Autenticación

CP-UN-001: Registro de Usuario

Campo	Detalle
ID	CP-UN-001
Descripción	Verificar que el registro de usuario funcione correctamente
Precondiciones	• Base de datos disponible - Email no registrado previamente
Técnica	Caja Blanca
Nivel	Unitaria
Datos de Entrada	{ "nombre": "Juan", "email": "juan@test.com", "password": "Pass123!" }
Resultado Esperado	• Status: 201 - Usuario creado en BD - Password hasheado - Token JWT generado
Resultado Obtenido	☑ PASS / ✕ FAIL
Observaciones	

CP-UN-002: Login con Credenciales Válidas

Campo	Detalle
ID	CP-UN-002
Descripción	Verificar autenticación con credenciales correctas
Precondiciones	• Usuario registrado en BD
Técnica	Caja Negra
Nivel	Unitaria

Campo	Detalle
Datos de Entrada	{ "email": "juan@test.com", "password": "Pass123!" }
Resultado Esperado	• Status: 200 - Token JWT válido - Datos del usuario
Resultado Obtenido	
Observaciones	

CP-UN-003: Login con Credenciales Inválidas

Campo	Detalle
ID	CP-UN-003
Descripción	Verificar rechazo de credenciales incorrectas
Precondiciones	• Usuario registrado en BD
Técnica	Caja Negra
Nivel	Unitaria
Datos de Entrada	{ "email": "juan@test.com", "password": "incorrecta" }
Resultado Esperado	• Status: 401 - Mensaje: "Credenciales inválidas"
Resultado Obtenido	
Observaciones	

Módulo de Tareas

CP-UN-004: Crear Tarea

Campo	Detalle
ID	CP-UN-004
Descripción	Verificar creación de nueva tarea
Precondiciones	• Usuario autenticado - Token válido

Campo	Detalle
Técnica	Caja Blanca
Nivel	Unitaria
Datos de Entrada	{ "titulo": "Tarea 1", "descripcion": "Descripción", "prioridad": "alta", "fechaVencimiento": "2024-12-31" }
Resultado Esperado	• Status: 201 - Tarea creada con ID - Estado inicial: "pendiente"
Resultado Obtenido	
Observaciones	

CP-UN-005: Listar Tareas del Usuario

Campo	Detalle
ID	CP-UN-005
Descripción	Obtener todas las tareas del usuario autenticado
Precondiciones	• Usuario autenticado - Al menos 1 tarea creada
Técnica	Caja Negra
Nivel	Unitaria
Datos de Entrada	Token JWT en header
Resultado Esperado	• Status: 200 - Array de tareas - Solo tareas del usuario
Resultado Obtenido	
Observaciones	

CP-UN-006: Actualizar Tarea

Campo	Detalle
ID	CP-UN-006
Descripción	Modificar datos de una tarea existente
Precondiciones	• Usuario autenticado - Tarea existe y pertenece al usuario

Campo	Detalle
Técnica	Caja Gris
Nivel	Unitaria
Datos de Entrada	{ "titulo": "Tarea Actualizada", "estado": "en_progreso" }
Resultado Esperado	• Status: 200 - Tarea actualizada - Timestamp de modificación
Resultado Obtenido	
Observaciones	

CP-UN-007: Eliminar Tarea

Campo	Detalle
ID	CP-UN-007
Descripción	Eliminar una tarea existente
Precondiciones	• Usuario autenticado - Tarea existe
Técnica	Caja Negra
Nivel	Unitaria
Datos de Entrada	ID de tarea: 123
Resultado Esperado	• Status: 200 - Tarea eliminada de BD - Mensaje de confirmación
Resultado Obtenido	
Observaciones	

CP-UN-008: Validación de Campos Obligatorios

Campo	Detalle
ID	CP-UN-008
Descripción	Verificar validación de campos requeridos

Campo	Detalle
Precondiciones	• Usuario autenticado
Técnica	Caja Blanca
Nivel	Unitaria
Datos de Entrada	{ "descripcion": "Sin título" }
Resultado Esperado	• Status: 400 - Mensaje: "El título es obligatorio"
Resultado Obtenido	
Observaciones	

Pruebas de integración

CP-INT-001: Flujo Completo Registro-Login-Crear Tarea

Campo	Detalle
ID	CP-INT-001
Descripción	Verificar integración entre módulos de autenticación y tareas
Precondiciones	• Sistema iniciado - BD limpia
Técnica	Caja Gris
Nivel	Integración
Pasos	1. Registrar usuario 2. Login 3. Crear tarea con token 4. Verificar tarea en BD
Resultado Esperado	• Todos los pasos exitosos - Tarea asociada al usuario correcto
Resultado Obtenido	

Campo	Detalle
Observaciones	

CP-INT-002: Middleware de Autenticación

Campo	Detalle
ID	CP-INT-002
Descripción	Verificar que el middleware protege rutas privadas
Precondiciones	• Servidor activo
Técnica	Caja Gris
Nivel	Integración
Datos de Entrada	Request sin token a /api/tareas
Resultado Esperado	• Status: 401 - Mensaje: "Token no proporcionado"
Resultado Obtenido	
Observaciones	

CP-INT-003: Integridad Referencial Usuario-Tareas

Campo	Detalle
ID	CP-INT-003
Descripción	Verificar relación entre usuarios y tareas en BD
Precondiciones	• Usuario con tareas creadas
Técnica	Caja Blanca
Nivel	Integración
Pasos	1. Crear usuario 2. Crear 3 tareas 3. Eliminar usuario 4. Verificar tareas
Resultado Esperado	• Tareas eliminadas en cascada o - Error de integridad referencial
Resultado Obtenido	
Observaciones	

CP-INT-004: Manejo de Transacciones

Campo	Detalle
ID	CP-INT-004
Descripción	Verificar rollback en caso de error
Precondiciones	• BD con transacciones habilitadas
Técnica	Caja Blanca
Nivel	Integración
Pasos	1. Iniciar transacción 2. Crear tarea 3. Forzar error 4. Verificar rollback
Resultado Esperado	• Tarea no creada en BD - Estado consistente
Resultado Obtenido	
Observaciones	

Pruebas de sistema

CP-SIS-001: Flujo Completo de Usuario

Campo	Detalle
ID	CP-SIS-001
Descripción	Simular uso completo del sistema
Precondiciones	• Sistema desplegado - BD operativa
Técnica	Caja Negra
Nivel	Sistema
Pasos	1. Registro 2. Login 3. Crear 5 tareas 4. Actualizar 2 5. Eliminar 1 6. Listar todas 7. Logout
Resultado Esperado	• Todas las operaciones exitosas - 4 tareas finales en BD
Resultado Obtenido	
Observaciones	

CP-SIS-002: Persistencia de Datos

Campo	Detalle
ID	CP-SIS-002
Descripción	Verificar que los datos persisten tras reinicio
Precondiciones	• Tareas creadas
Técnica	Caja Negra
Nivel	Sistema
Pasos	1. Crear tareas 2. Reiniciar servidor 3. Consultar tareas
Resultado Esperado	• Tareas disponibles después del reinicio
Resultado Obtenido	
Observaciones	

CP-SIS-003: Manejo de Errores del Sistema

Campo	Detalle
ID	CP-SIS-003
Descripción	Verificar respuesta ante fallos de BD
Precondiciones	• Sistema activo
Técnica	Caja Gris
Nivel	Sistema
Pasos	1. Detener BD 2. Intentar crear tarea 3. Verificar respuesta
Resultado Esperado	• Status: 503 - Mensaje: "Servicio no disponible" - No crash del servidor
Resultado Obtenido	
Observaciones	

Pruebas funcionales

CP-ACE-001: Usuario Puede Gestionar Sus Tareas

Campo	Detalle
ID	CP-ACE-001
Descripción	Criterio de aceptación: CRUD completo de tareas
Precondiciones	• Usuario registrado
Técnica	Caja Negra
Nivel	Aceptación
Criterios	• Crear tarea ✓ - Ver tareas ✓ - Editar tarea ✓ - Eliminar tarea ✓
Resultado Esperado	• Todas las operaciones funcionales - Interfaz API clara

Campo	Detalle
Resultado Obtenido	
Observaciones	

CP-ACE-002: Seguridad de Datos del Usuario

Campo	Detalle
ID	CP-ACE-002
Descripción	Usuario solo ve sus propias tareas
Precondiciones	• 2 usuarios con tareas
Técnica	Caja Negra
Nivel	Aceptación
Pasos	1. Login usuario A 2. Listar tareas 3. Verificar que no aparecen tareas de usuario B
Resultado Esperado	• Solo tareas propias visibles - No acceso a tareas ajenas
Resultado Obtenido	
Observaciones	

CP-ACE-003: Filtrado y Búsqueda de Tareas

Campo	Detalle
ID	CP-ACE-003
Descripción	Usuario puede filtrar tareas por estado/prioridad

Campo	Detalle
Precondiciones	• Usuario con múltiples tareas
Técnica	Caja Negra
Nivel	Aceptación
Datos de Entrada	GET /api/tareas?estado=completada&prioridad=alta
Resultado Esperado	• Solo tareas que cumplen criterios - Respuesta ordenada
Resultado Obtenido	
Observaciones	

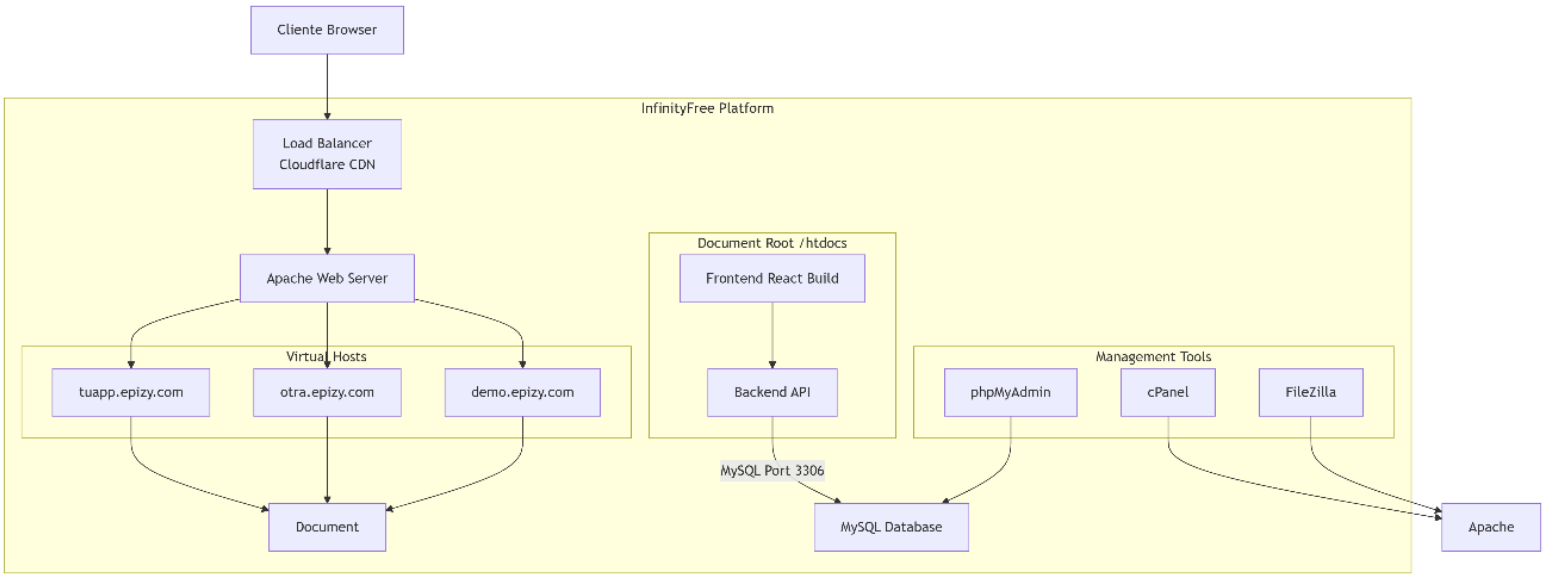
Pruebas no funcionales

Pruebas de Rendimiento

CP-REN-001: Tiempo de Respuesta de Endpoints

Campo	Detalle
ID	CP-REN-001
Descripción	Medir tiempo de respuesta promedio
Precondiciones	• Sistema en producción
Técnica	Caja Negra
Tipo	Rendimiento
Métrica	Tiempo de respuesta < 200ms
Herramienta	Apache JMeter / Artillery
Resultado Esperado	• GET /tareas: < 150ms

Proceso de despliegue detallado



FASE 1: Preparación local.

Paso	Acción	Comando/Detalle
1.1	Build Frontend	`npm run build`
1.2	Verificar build	Revisar carpeta `/build`
1.3	Configurar API URL	Editar `config.js` con URL producción
1.4	Preparar Backend PHP	Adaptar endpoints para InfinityFree

FASE 2: Configuración InfinityFree.

Paso	Acción	Ubicación
2.1	Crear cuenta	infinityfree.net/signup
2.2	Crear hosting	Dashboard > Create Account
2.3	Elegir dominio	tuapp.epizy.com
2.4	Obtener credenciales	Account Details

FASE 3: Base de datos.

Paso	Acción	Herramienta
3.1	Crear database	VistaPanel > MySQL
3.2	Importar SQL	phpMyAdmin
3.3	Verificar tablas	phpMyAdmin > Structure

FASE 4: Subida de archivos.

Paso	Acción	Herramienta
4.1	Conectar FTP	FileZilla
4.2	Subir frontend	/htdocs/
4.3	Subir backend	/htdocs/api/
4.4	Configurar .htaccess	File Manager

FASE 5: Verificación.

Paso	Acción	Resultado Esperado
5.1	Acceder URL	Página carga correctamente
5.2	Probar login	Autenticación funciona
5.3	Probar CRUD	Tareas se crean/editan/eliminan
5.4	Revisar consola	Sin errores JavaScript